

Clustering program

Generated by Doxygen 1.8.17

1 analyze_residues.f90	1
1.1 Description	1
1.2 Parser description	1
1.3 Required Arguments	1
1.4 Optional arguments	1
1.4.1 Modules Used	1
1.4.2 Main Routines Called	2
1.4.3 Output	2
1.4.4 Usage Example	2
1.4.5 Author	2
2 clust.f90	3
2.1 Description	3
2.2 Parser description	3
2.3 Required Arguments	3
2.4 Optional arguments	3
2.4.1 Modules Used	4
2.4.2 Main Routines Called	4
2.4.3 Notes	4
2.4.4 Usage Example	4
2.4.5 Author	4
3 clust_all.f90	5
3.1 Description	5
3.2 Parser description	5
3.3 Required Arguments	5
3.4 Optional arguments	5
3.4.1 Modules Used	6
3.4.2 Workflow Summary	6
3.4.3 Usage	6
3.4.4 Author	6
4 make_data.f90	7
4.1 Description	7
4.2 Parser description	7
4.3 Required Arguments	7
4.4 Optional arguments	7
4.4.1 Modules Used	8
4.4.2 Usage	8
4.4.3 Author	8
5 threshold.f90	9
5.1 Description	9
5.2 Parser description	9

5.3 Required Arguments	9
5.4 Optional arguments	10
5.4.1 Modules Used	10
5.4.2 Main Routines Called	10
5.4.3 Usage Example	10
5.4.4 Author	10
6 Fortran Unit Tests (<tt>src/unit_test</tt>)	11
6.1 Included Test Programs	11
6.2 Current Coverage Notes	11
6.3 Default Test Execution Paths	12
6.3.1 From <tt>src/</tt> (Makefile target)	12
6.3.2 From <tt>src/unit_test/</tt> (script)	12
6.4 Author	12
7 Modules Index	13
7.1 Modules List	13
8 Data Type Index	15
8.1 Data Types List	15
9 File Index	17
9.1 File List	17
10 Module Documentation	19
10.1 maths Module Reference	19
10.1.1 Function/Subroutine Documentation	19
10.1.1.1 calculate_cog()	19
10.1.1.2 calculate_distance()	20
10.1.1.3 cross()	20
10.1.1.4 rmsd()	20
10.1.1.5 update_complex()	21
10.1.1.6 vectors_angle_2d()	21
10.1.1.7 vectors_angle_3d()	22
10.2 mod_array Module Reference	22
10.2.1 Function/Subroutine Documentation	23
10.2.1.1 array_angle()	23
10.2.1.2 array_atoms_dist()	24
10.2.1.3 array_z_coord()	24
10.2.1.4 merge_sorted_segments()	25
10.2.1.5 read_array()	25
10.2.1.6 write_array()	26
10.3 mod_assoc Module Reference	26
10.3.1 Function/Subroutine Documentation	26

10.3.1.1 allocate_assoc_object()	27
10.3.1.2 fill_assoc_object()	27
10.3.1.3 size_assoc()	27
10.3.1.4 write_complexes()	28
10.4 mod_clust_algorithm Module Reference	28
10.4.1 Function/Subroutine Documentation	28
10.4.1.1 linkage_clustering_from_array()	28
10.4.1.2 linkage_clustering_from_matrix()	29
10.4.1.3 sort_complexes()	29
10.4.1.4 write_clust_info()	30
10.4.1.5 write_cluster_complexes()	30
10.4.1.6 write_cluster_elements()	31
10.5 mod_matrix Module Reference	31
10.5.1 Function/Subroutine Documentation	32
10.5.1.1 matrix_rmsd()	32
10.5.1.2 read_matrix()	32
10.5.1.3 write_matrix()	33
10.6 mod_pdb Module Reference	33
10.6.1 Function/Subroutine Documentation	33
10.6.1.1 allocate_pdb_object()	34
10.6.1.2 count_atoms()	34
10.6.1.3 count_chains()	34
10.6.1.4 count_residues()	35
10.6.1.5 fill_pdb_object()	35
10.7 mod_threshold Module Reference	36
10.7.1 Function/Subroutine Documentation	36
10.7.1.1 array_angle()	36
10.7.1.2 array_atoms_dist()	37
10.7.1.3 array_z_coord()	37
10.7.1.4 sort_array()	38
10.8 read_input Module Reference	38
10.8.1 Function/Subroutine Documentation	38
10.8.1.1 read_assoc()	39
10.8.1.2 read_pdb()	39
11 Data Type Documentation	41
11.1 mod_assoc::type_assoc_file Type Reference	41
11.1.1 Detailed Description	41
11.1.2 Member Data Documentation	41
11.1.2.1 head	42
11.1.2.2 lines	42
11.1.2.3 nlines	42

11.1.2.4 rot1	42
11.1.2.5 rot2	42
11.1.2.6 trans_vector	42
11.1.2.7 xc1	42
11.1.2.8 xc2	43
11.2 mod_pdb::type_pdb_atom Type Reference	43
11.2.1 Member Data Documentation	43
11.2.1.1 beta	43
11.2.1.2 chainid	43
11.2.1.3 coord	43
11.2.1.4 id	44
11.2.1.5 name	44
11.2.1.6 occ	44
11.2.1.7 record	44
11.2.1.8 resid	44
11.2.1.9 resname	44
11.3 mod_pdb::type_pdb_chain Type Reference	44
11.4 mod_pdb::type_pdb_file Type Reference	45
11.4.1 Detailed Description	45
11.4.2 Member Data Documentation	45
11.4.2.1 atoms	45
11.4.2.2 chains	45
11.4.2.3 natoms	45
11.4.2.4 nchains	46
11.4.2.5 nresidues	46
11.4.2.6 residues	46
11.5 mod_pdb::type_pdb_residue Type Reference	46
11.5.1 Member Data Documentation	46
11.5.1.1 atoms	47
11.5.1.2 chainid	47
11.5.1.3 natoms	47
11.5.1.4 resid	47
11.5.1.5 resname	47
12 File Documentation	49
12.1 analyze_residues.f90 File Reference	49
12.1.1 Detailed Description	49
12.1.2 Function/Subroutine Documentation	50
12.1.2.1 main()	50
12.1.2.2 print_help()	50
12.2 analyze_residues.md File Reference	50
12.3 clust.f90 File Reference	50

12.3.1 Detailed Description	50
12.3.2 Function/Subroutine Documentation	51
12.3.2.1 main()	51
12.3.2.2 parse_arguments()	51
12.3.2.3 print_help()	51
12.4 clust.md File Reference	52
12.5 clust_all.f90 File Reference	52
12.5.1 Detailed Description	52
12.5.2 Function/Subroutine Documentation	52
12.5.2.1 clust_all()	53
12.5.2.2 parse_arguments()	53
12.5.2.3 print_help()	54
12.5.2.4 read_atoms_coord()	54
12.6 clust_all.md File Reference	55
12.7 make_data.f90 File Reference	55
12.7.1 Detailed Description	55
12.7.2 Function/Subroutine Documentation	55
12.7.2.1 main()	56
12.7.2.2 parse_arguments()	56
12.7.2.3 print_help()	57
12.7.2.4 read_atoms_coord()	57
12.8 make_data.md File Reference	57
12.9 maths.f90 File Reference	57
12.9.1 Detailed Description	58
12.10 mod_array.f90 File Reference	58
12.10.1 Detailed Description	59
12.11 mod_assoc.f90 File Reference	59
12.11.1 Detailed Description	60
12.12 mod_clust_algorithm.f90 File Reference	60
12.12.1 Detailed Description	61
12.13 mod_matrix.f90 File Reference	61
12.13.1 Detailed Description	62
12.14 mod_pdb.f90 File Reference	62
12.14.1 Detailed Description	63
12.15 mod_threshold.f90 File Reference	63
12.15.1 Detailed Description	64
12.16 read_input.f90 File Reference	64
12.16.1 Detailed Description	64
12.17 threshold.f90 File Reference	65
12.17.1 Detailed Description	65
12.17.2 Function/Subroutine Documentation	65
12.17.2.1 main()	66

12.17.2.2	parse_arguments()	66
12.17.2.3	print_help()	67
12.17.2.4	read_atoms_coord()	67
12.17.2.5	write_array()	68
12.17.2.6	write_cutoff_complexes()	68
12.18	threshold.md File Reference	68
12.19	unit_test/README.md File Reference	68
12.20	unit_test/run_all_tests.sh File Reference	68
12.21	unit_test/test_array.f90 File Reference	68
12.21.1	Function/Subroutine Documentation	69
12.21.1.1	test_array()	69
12.21.1.2	test_array_angle_sorting()	69
12.21.1.3	test_array_atoms_dist_sorting()	70
12.21.1.4	test_array_z_coord_sorting()	70
12.21.1.5	test_merge_sorted_segments()	71
12.21.1.6	test_write_read_array()	71
12.22	unit_test/test_assoc.f90 File Reference	72
12.22.1	Function/Subroutine Documentation	72
12.22.1.1	test_allocate_assoc_object()	72
12.22.1.2	test_assoc()	73
12.22.1.3	test_read_assoc_file()	74
12.22.1.4	test_size_assoc()	75
12.23	unit_test/test_clustering.f90 File Reference	75
12.23.1	Detailed Description	76
12.23.2	Function/Subroutine Documentation	76
12.23.2.1	test_cluster_writing()	76
12.23.2.2	test_clustering()	77
12.23.2.3	test_clustering_statistics()	77
12.23.2.4	test_maximum_linkage()	77
12.23.2.5	test_mean_linkage()	78
12.23.2.6	test_minimum_linkage()	78
12.23.2.7	test_simple_clustering()	78
12.24	unit_test/test_maths.f90 File Reference	79
12.24.1	Function/Subroutine Documentation	79
12.24.1.1	assert_vector_equal()	79
12.24.1.2	test_calculate_cog()	79
12.24.1.3	test_calculate_distance()	80
12.24.1.4	test_cross_product()	80
12.24.1.5	test_maths()	81
12.24.1.6	test_rmsd()	83
12.24.1.7	test_update_complex()	83
12.24.1.8	test_vectors_angle_2d()	84

12.24.1.9 test_vectors_angle_3d()	84
12.25 unit_test/test_matrix.f90 File Reference	85
12.25.1 Function/Subroutine Documentation	85
12.25.1.1 test_matrix()	85
12.25.1.2 test_matrix_rmsd_calc()	86
12.25.1.3 test_write_read_matrix()	86
12.26 unit_test/test_pdb.f90 File Reference	87
12.26.1 Function/Subroutine Documentation	87
12.26.1.1 test_atom_properties()	87
12.26.1.2 test_chain_management()	88
12.26.1.3 test_mod_pdb_routines()	88
12.26.1.4 test_pdb()	89
12.26.1.5 test_pdb_allocation()	90
12.26.1.6 test_residue_management()	90
12.27 unit_test/test_read_input.f90 File Reference	91
12.27.1 Function/Subroutine Documentation	91
12.27.1.1 test_error_handling()	91
12.27.1.2 test_read_input()	92
12.27.1.3 test_read_input_assoc()	93
12.28 unit_test/test_threshold.f90 File Reference	93
12.28.1 Function/Subroutine Documentation	93
12.28.1.1 test_array_angle()	94
12.28.1.2 test_array_atoms_dist()	94
12.28.1.3 test_array_z_coord()	95
12.28.1.4 test_sort_array()	95
12.28.1.5 test_threshold()	96
Index	97

Chapter 1

analyze_residues.f90

1.1 Description

`analyze_residues` evaluates residue proximity to a surface across encounter complexes.

For each residue in `pdb2`, it computes a center-of-geometry, applies encounter transformations, and records encounters where the transformed residue position is below a distance threshold.

1.2 Parser description

1.3 Required Arguments

- `-pdb2 <file>`
- `-complexes <file>`

1.4 Optional arguments

- `-nb_encounters <N>` (default: all)
- `-threshold <value>` (default: 6.0)
- `-help`

1.4.1 Modules Used

- `read_input`
- `maths`

1.4.2 Main Routines Called

- read_pdb (from `read_input`)
- read_assoc (from `read_input`)
- calculate_cog (from `maths`)
- update_complex (from `maths`)
- print_help (internal)

1.4.3 Output

- `closest_residues.txt`: each residue line includes encounter indexes that satisfy the threshold condition.

1.4.4 Usage Example

```
./analyze_residues -pdb2 p2_noh.pdb -complexes assoc_complexes -nb_encounters 5000 -threshold 6.0
```

1.4.5 Author

Abraham Muñoz-Chicharro

Chapter 2

clust.f90

2.1 Description

`clust` performs hierarchical clustering on encounter data using `min`, `max`, or `mean` linkage.

The input passed with `-input` is auto-detected:

- single line → treated as a 1D array
- multiple lines → treated as a 2D distance matrix

The program reads encounter complexes, runs clustering, and writes output files using the provided output name.

2.2 Parser description

2.3 Required Arguments

- `-complexes <file>`
- `-input <file>`
- `-output_name <name>`

2.4 Optional arguments

- `-nb_encounters <N>` (default: all)
- `-linkage <min|max|mean>` (default: mean)
- `-help`

2.4.1 Modules Used

- `mod_matrix`
- `mod_array`
- `mod_clust_algorithm`
- `read_input`

2.4.2 Main Routines Called

- `read_assoc` (from `read_input`)
- `read_array` (from `mod_array`)
- `read_matrix` (from `mod_matrix`)
- `linkage_clustering_from_array` (from `mod_clust_algorithm`)
- `linkage_clustering_from_matrix` (from `mod_clust_algorithm`)
- `print_help` (internal)

2.4.3 Notes

- Clustering runs on CPU for both array and matrix inputs.
- If `-nb_encounters` is omitted, all encounters in the complexes file are used.

2.4.4 Usage Example

```
./clust -complexes assoc_complexes -input matrix_z.txt -nb_encounters 5000 -output_name Cu_z -linkage mean
```

2.4.5 Author

Abraham Muñiz-Chicharro

Chapter 3

clust_all.f90

3.1 Description

`clust_all` builds array/matrix data in memory from structural and encounter inputs, then performs hierarchical clustering in a single workflow.

It is designed to avoid intermediate disk I/O for data construction steps while still writing clustering outputs.

3.2 Parser description

3.3 Required Arguments

- `-pdb2 <file>`
- `-complexes <file>`
- `-atoms2 <atom ...>` (or `-atoms2a`)
- `-data_type <type>`
- `-datadist <file>`

3.4 Optional arguments

- `-pdb1 <file>` (required for `atoms_dist` and `angle` types)
- `-atoms1, -atoms1a, -atoms1b, -atoms2b` (as needed by data type)
- `-nb_encounters <N>`
- `-linkage <min|max|mean>` (default: `mean`)
- `-output_name <name>`
- `-help`

3.4.1 Modules Used

- `read_input`
- `mod_matrix`
- `mod_array`
- `mod_clust_algorithm`
- `mod_assoc`

3.4.2 Workflow Summary

1. Parse CLI options and validate required inputs.
2. Read PDB and complexes files.
3. Build requested data representation according to `-data_type`.
4. Run clustering with selected linkage method.
5. Write clustering outputs using `-output_name`.

3.4.3 Usage

```
./clust_all -help  
./clust_all -data_type rmsd -help
```

3.4.4 Author

Abraham Muñiz-Chicharro

Chapter 4

make_data.f90

4.1 Description

`make_data` generates encounter-derived numeric data and writes it to disk.

The selected mode is controlled by `-data_type`:

- `rmsd`
- `z_coord`
- `atoms_dist`
- `2D_angle`
- `3D_angle`

4.2 Parser description

4.3 Required Arguments

- `-pdb2 <file>`
- `-atoms2 <atom ...>` (or `-atoms2a`)
- `-complexes <file>`
- `-data_type <type>`
- `-input <file>`

4.4 Optional arguments

- `-pdb1 <file>` (required for `atoms_dist` and angle types)
- `-atoms1, -atoms1a, -atoms1b, -atoms2b` (as needed by data type)
- `-nb_encounters <N>`
- `-help`

4.4.1 Modules Used

- `read_input`
- `mod_matrix`
- `mod_array`
- `mod_assoc`

4.4.2 Usage

```
./make_data -help  
./make_data -data_type rmsd -help
```

4.4.3 Author

Abraham Muñiz-Chicharro

Chapter 5

threshold.f90

5.1 Description

`threshold` selects encounter complexes based on a threshold criterion and writes both the computed value array and the filtered complexes.

Supported threshold modes include:

- `z_coord`
- `atoms_dist`
- `2D_angle`
- `3D_angle`

Type-specific help can be shown with:

```
./threshold -threshold_type <type> -help
```

5.2 Parser description

5.3 Required Arguments

- `-pdb2 <file>`
- `-atoms2 <atom ...> (or -atoms2a)`
- `-array <file>`
- `-complexes <file>`
- `-complexes_output <file>`
- `-threshold_type <type>`
- `-cutoff <value>`

5.4 Optional arguments

- `-pdb1 <file>`
- `-atoms1, -atoms1a, -atoms1b, -atoms2b`
- `-nb_encounters <N>`
- `-help`

5.4.1 Modules Used

- `read_input`
- `mod_threshold`

5.4.2 Main Routines Called

- `read_pdb` (from `read_input`)
- `read_assoc` (from `read_input`)
- `array_z_coord, array_angle, array_atoms_dist` (from `mod_threshold`)
- `sort_array` (from `mod_threshold`)
- `write_array, write_cutoff_complexes` (internal)
- `print_help` (internal)

5.4.3 Usage Example

```
./threshold -pdb2 p2_noh.pdb -atoms2 Cu -complexes assoc_complexes -threshold_type z_coord -complexes_output  
threshold_z.txt -array z_values.txt -cutoff 5.0  
./threshold -threshold_type atoms_dist -help
```

5.4.4 Author

Abraham Muñoz-Chicharro

Chapter 6

Fortran Unit Tests (<tt>src/unit_test</tt>)

This directory contains standalone Fortran test programs for core clustering modules.

6.1 Included Test Programs

- `test_maths.f90`: numerical/geometry operations in `maths`
- `test_matrix.f90`: matrix I/O and RMSD matrix operations in `mod_matrix`
- `test_array.f90`: array I/O, array-based encounter metrics, and array/metadata reordering in `mod_array`
- `test_threshold.f90`: threshold operations in `mod_threshold`
- `test_pdb.f90`: PDB parsing/structures in `mod_pdb`
- `test_clustering.f90`: clustering logic in `mod_clust_algorithm`
- `test_assoc.f90`: association file handling in `mod_assoc`
- `test_read_input.f90`: integrated input loading in `read_input`

6.2 Current Coverage Notes

- `test_matrix.f90` currently covers:
 - `write_matrix()` / `read_matrix()`
 - `matrix_rmsd()`
- `test_array.f90` currently covers:
 - `write_array()` / `read_array()`
 - `array_z_coord()`
 - `array_atoms_dist()`
 - `array_angle()`
 - `merge_sorted_segments()`
- `test_threshold.f90` covers threshold-oriented array calculations in `mod_threshold`.

6.3 Default Test Execution Paths

6.3.1 From <tt>src/</tt> (Makefile target)

```
make test
```

This is the authoritative default path. It runs the standard Fortran test set defined in `src/Makefile`, including `test_array`.

6.3.2 From <tt>src/unit_test/</tt> (script)

```
bash run_all_tests.sh
```

This script compiles/runs the list currently defined in `run_all_tests.sh`.

6.4 Author

Abraham Muñiz-Chicharro

Chapter 7

Modules Index

7.1 Modules List

Here is a list of all modules with brief descriptions:

maths	19
mod_array	22
mod_assoc	26
mod_clust_algorithm	28
mod_matrix	31
mod_pdb	33
mod_threshold	36
read_input	38

Chapter 8

Data Type Index

8.1 Data Types List

Here are the data types with brief descriptions:

mod_assoc::type_assoc_file	
Data type for storing binned data (eg	41
mod_pdb::type_pdb_atom	43
mod_pdb::type_pdb_chain	44
mod_pdb::type_pdb_file	
Data type for storing binned data (eg	45
mod_pdb::type_pdb_residue	46

Chapter 9

File Index

9.1 File List

Here is a list of all files with brief descriptions:

analyze_residues.f90	Residue proximity analysis across transformed encounters	49
clust.f90	Hierarchical clustering driver for encounter matrices/arrays	50
clust_all.f90	Build matrices/arrays and perform clustering in memory (no disk I/O). Usage: ./clust_all -help	52
make_data.f90	Build encounter-based matrix/array data from structural inputs	55
maths.f90	Mathematical and geometric utilities for clustering workflows	57
mod_array.f90	Build/read/write encounter-based arrays and per-encounter metrics	58
mod_assoc.f90	Association/complexes file data structures and parsing routines	59
mod_clust_algorithm.f90	Hierarchical clustering algorithms and output helpers	60
mod_matrix.f90	Build/read/write encounter-based matrices and arrays	61
mod_pdb.f90	PDB data structures and parsing/allocation routines	62
mod_threshold.f90	Utilities to compute threshold arrays from encounter complexes	63
read_input.f90	Input readers for PDB and association/complexes files	64
threshold.f90	Threshold-based encounter selection utility	65
unit_test/run_all_tests.sh		68
unit_test/test_array.f90		68
unit_test/test_assoc.f90		72
unit_test/test_clustering.f90	Unit tests for clustering algorithms in mod_clust_algorithm	75
unit_test/test_maths.f90		79
unit_test/test_matrix.f90		85
unit_test/test_pdb.f90		87
unit_test/test_read_input.f90		91
unit_test/test_threshold.f90		93

Chapter 10

Module Documentation

10.1 maths Module Reference

Functions/Subroutines

- subroutine `cross` (v1, v2, v3)
Compute the cross product of two 3D vectors.
- subroutine `update_complex` (xc1, xc2, trans, rot_vx, rot_vy, nb_atoms, coords, new_coords)
Apply translation and rotation to a set of atomic coordinates.
- subroutine `vectors_angle_3d` (vector1, vector2, theta_degrees)
Compute the angle in degrees between two 3D vectors.
- subroutine `vectors_angle_2d` (vector1, vector2, theta_degrees)
Compute the angle in degrees between two vectors projected to 2D.
- subroutine `rmsd` (rmsd_value, nb_atoms, coords1, coords2)
Compute the root-mean-square deviation (RMSD) between two sets of coordinates.
- subroutine `calculate_cog` (cog, coords, nb_atoms)
Compute the center of geometry (centroid) of a set of points.
- subroutine `calculate_distance` (p1, p2, dist)
Compute Euclidean distance between two 3D points.

10.1.1 Function/Subroutine Documentation

10.1.1.1 `calculate_cog()`

```
subroutine maths::calculate_cog (  
    real (kind=8), dimension(3), intent(out) cog,  
    real (kind=8), dimension(nb_atoms, 3), intent(in) coords,  
    integer, intent(in) nb_atoms )
```

Compute the center of geometry (centroid) of a set of points.

Sums the coordinates and divides by the number of points to produce the centroid.

Parameters

out	<i>cog</i>	Centroid vector (3 elements)
in	<i>coords</i>	Input coordinates, shape (nb_atoms,3)
in	<i>nb_atoms</i>	Number of atoms / points

10.1.1.2 calculate_distance()

```

subroutine maths::calculate_distance (
    real (kind=8), dimension(3), intent(in) p1,
    real (kind=8), dimension(3), intent(in) p2,
    real (kind=8), intent(out) dist )

```

Compute Euclidean distance between two 3D points.

Parameters

in	<i>p1</i>	First point (3 elements)
in	<i>p2</i>	Second point (3 elements)
out	<i>dist</i>	Euclidean distance $\ p1 - p2\ $

10.1.1.3 cross()

```

subroutine maths::cross (
    real ( kind=8 ), dimension ( 3 ), intent(in) v1,
    real ( kind=8 ), dimension ( 3 ), intent(in) v2,
    real ( kind=8 ), dimension ( 3 ), intent(out) v3 )

```

Compute the cross product of two 3D vectors.

Computes $v3 = v1 \times v2$.

Parameters

in	<i>v1</i>	Left-hand operand vector (3 elements)
in	<i>v2</i>	Right-hand operand vector (3 elements)
out	<i>v3</i>	Resulting cross product (3 elements)

10.1.1.4 rmsd()

```

subroutine maths::rmsd (
    real(kind=8), intent(out) rmsd_value,

```

```
integer, intent(in) nb_atoms,
real(kind=8), dimension(:, :), intent(in) coords1,
real(kind=8), dimension(:, :), intent(in) coords2 )
```

Compute the root-mean-square deviation (RMSD) between two sets of coordinates.

Calculates $\text{RMSD} = \sqrt{(1/N) * \sum_i ||\text{coords1}(i,:) - \text{coords2}(i,:)||^2}$.

Parameters

out	<i>rmsd_value</i>	Resulting RMSD (scalar)
in	<i>nb_atoms</i>	Number of atoms / points (integer)
in	<i>coords1</i>	First coordinate set, shape (nb_atoms,3)
in	<i>coords2</i>	Second coordinate set, shape (nb_atoms,3)

10.1.1.5 update_complex()

```
subroutine maths::update_complex (
    real ( kind = 8 ), dimension ( 3 ), intent(in) xc1,
    real ( kind = 8 ), dimension ( 3 ), intent(in) xc2,
    real ( kind = 8 ), dimension ( 3 ), intent(in) trans,
    real ( kind = 8 ), dimension ( 3 ), intent(in) rot_vx,
    real ( kind = 8 ), dimension ( 3 ), intent(in) rot_vy,
    integer, intent(in) nb_atoms,
    real ( kind = 8 ), dimension (:, :), intent(in) coords,
    real ( kind = 8 ), dimension (:, :), intent(out) new_coords )
```

Apply translation and rotation to a set of atomic coordinates.

Transforms the coordinates of a complex. The routine first recenters coordinates to the origin with *xc2*, applies a rotation defined by *rot_vx* and *rot_vy* (their cross product defines the third axis), then applies an additional translation *trans*, moves to the new frame centered at *xc1*, and writes the results to *new_coords*.

Parameters

in	<i>xc1</i>	Target center position (3)
in	<i>xc2</i>	Original center position (3)
in	<i>trans</i>	Translation to apply after rotation (3)
in	<i>rot_vx</i>	First rotation axis vector (3)
in	<i>rot_vy</i>	Second rotation axis vector (3)
in	<i>nb_atoms</i>	Number of atoms (integer)
in	<i>coords</i>	Input coordinates, shape (nb_atoms,3)
out	<i>new_coords</i>	Output coordinates, shape (nb_atoms,3)

10.1.1.6 vectors_angle_2d()

```
subroutine maths::vectors_angle_2d (
```

```

real( kind=8 ), dimension( 3 ), intent(in) vector1,
real( kind=8 ), dimension( 3 ), intent(in) vector2,
real (kind=8), intent(out) theta_degrees )

```

Compute the angle in degrees between two vectors projected to 2D.

Uses only the first two components of the input vectors to compute the angle in the XY plane. Returns 0 if either vector has zero length in the XY plane.

Parameters

in	<i>vector1</i>	First vector (use components 1 and 2)
in	<i>vector2</i>	Second vector (use components 1 and 2)
out	<i>theta_degrees</i>	Angle between projected vectors in degrees

10.1.1.7 vectors_angle_3d()

```

subroutine maths::vectors_angle_3d (
    real( kind=8 ), dimension( 3 ), intent(in) vector1,
    real( kind=8 ), dimension( 3 ), intent(in) vector2,
    real (kind=8), intent(out) theta_degrees )

```

Compute the angle in degrees between two 3D vectors.

Calculates the angle (in degrees) between *vector1* and *vector2* using the dot product formula and `acos`.

Parameters

in	<i>vector1</i>	First 3D vector (3 elements)
in	<i>vector2</i>	Second 3D vector (3 elements)
out	<i>theta_degrees</i>	Angle between vectors in degrees

10.2 mod_array Module Reference

Functions/Subroutines

- subroutine [array_z_coord](#) (complexes, array_sorted, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute←_crds)
Compute the Z-coordinate (along transformed z-axis) for each encounter and sort.
- subroutine [array_atoms_dist](#) (complexes, array_sorted, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute1_crds, solute2_crds)
Compute minimum inter-atomic distance between two solutes for each encounter and sort.
- subroutine [array_angle](#) (complexes, array_sorted, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, point1a, point1b, point2a, point2b, dimensions)
Compute angle between two vectors (defined by atom groups) per encounter and sort.
- subroutine [write_array](#) (array, filename)

Write a numeric array to a formatted text file.

- subroutine [read_array](#) (array, n, filename)

Read a numeric array from a formatted text file.

- subroutine [merge_sorted_segments](#) (arr, complexes, tmp_arr, tmp_comp, left, mid, right)

Merge two adjacent sorted segments of arrays in place.

10.2.1 Function/Subroutine Documentation

10.2.1.1 array_angle()

```
subroutine mod_array::array_angle (
    type(type_assoc_file), intent(inout) complexes,
    real(kind=8), dimension(n), intent(out) array_sorted,
    integer, intent(in) n,
    integer, intent(in) nb_atoms,
    real(kind=8), dimension(3), intent(in) xc1,
    real(kind=8), dimension(3), intent(in) xc2,
    real(kind=8), dimension(:, :), intent(in) trans_vector,
    real(kind=8), dimension(:, :), intent(in) rot1,
    real(kind=8), dimension(:, :), intent(in) rot2,
    real(kind=8), dimension(:), intent(in) point1a,
    real(kind=8), dimension(:), intent(in) point1b,
    real(kind=8), dimension(:), intent(in) point2a,
    real(kind=8), dimension(:), intent(in) point2b,
    integer, intent(in) dimensions )
```

Compute angle between two vectors (defined by atom groups) per encounter and sort.

Computes the angle between two vectors (defined by atom groups) for each encounter. The resulting array is sorted from minimum to maximum value.

Parameters

in, out	<i>complexes</i>	Association object reordered consistently with sorted outputs
out	<i>array_sorted</i>	Output array of length n with angles (sorted)
in	<i>n</i>	Number of encounters
in	<i>nb_atoms</i>	Number of atoms used for transformation
in	<i>xc1,xc2</i>	Solute centers used for translation transformations
in	<i>trans_vector</i>	Translation vectors for each encounter (n, 3)
in	<i>rot1,rot2</i>	Rotation vectors for each encounter (n, 3)
in	<i>point1a</i>	First point defining reference vector
in	<i>point1b</i>	Second point defining reference vector
in	<i>point2a</i>	First point of vector to be transformed
in	<i>point2b</i>	Second point of vector to be transformed
in	<i>dimensions</i>	Use 2 or 3 to select projection

10.2.1.2 array_atoms_dist()

```
subroutine mod_array::array_atoms_dist (
    type(type_assoc_file), intent(inout) complexes,
    real(kind=8), dimension(n), intent(out) array_sorted,
    integer, intent(in) n,
    integer, intent(in) nb_atoms,
    real(kind=8), dimension(3), intent(in) xc1,
    real(kind=8), dimension(3), intent(in) xc2,
    real(kind=8), dimension(:, :), intent(in) trans_vector,
    real(kind=8), dimension(:, :), intent(in) rot1,
    real(kind=8), dimension(:, :), intent(in) rot2,
    real(kind=8), dimension(:, :), intent(in) solute1_crds,
    real(kind=8), dimension(:, :), intent(in) solute2_crds )
```

Compute minimum inter-atomic distance between two solutes for each encounter and sort.

For each encounter, the second solute is transformed and the minimum distance to `solute1_crds` is computed and stored in the output array. The resulting array is sorted from minimum to maximum value.

Parameters

in, out	<i>complexes</i>	Association object reordered consistently with sorted outputs
out	<i>array_sorted</i>	Output array of length <i>n</i> with minimum distances (sorted)
in	<i>n</i>	Number of encounters
in	<i>nb_atoms</i>	Number of atoms in <i>solute2_crds</i>
in	<i>xc1,xc2</i>	Solute centers used for translation transformations
in	<i>trans_vector</i>	Translation vectors for each encounter (<i>n</i> , 3)
in	<i>rot1,rot2</i>	Rotation vectors for each encounter (<i>n</i> , 3)
in	<i>solute1_crds</i>	Coordinates of solute1 atoms
in	<i>solute2_crds</i>	Coordinates of solute2 atoms

10.2.1.3 array_z_coord()

```
subroutine mod_array::array_z_coord (
    type(type_assoc_file), intent(inout) complexes,
    real(kind=8), dimension(n), intent(out) array_sorted,
    integer, intent(in) n,
    integer, intent(in) nb_atoms,
    real(kind=8), dimension(3), intent(in) xc1,
    real(kind=8), dimension(3), intent(in) xc2,
    real(kind=8), dimension(:, :), intent(in) trans_vector,
    real(kind=8), dimension(:, :), intent(in) rot1,
    real(kind=8), dimension(:, :), intent(in) rot2,
    real(kind=8), dimension(:, :), intent(in) solute_crds )
```

Compute the Z-coordinate (along transformed z-axis) for each encounter and sort.

Computes the Z coordinate for each encounter and fills the output array. The resulting array is sorted from minimum to maximum value.

Parameters

in, out	<i>complexes</i>	Association object reordered consistently with sorted outputs
out	<i>array_sorted</i>	Output array of length <i>n</i> with z-coordinates (sorted)
in	<i>n</i>	Number of encounters
in	<i>nb_atoms</i>	Number of atoms in <i>solute_crds</i>
in	<i>xc1,xc2</i>	Solute centers used for translation transformations
in	<i>trans_vector</i>	Translation vectors for each encounter (<i>n</i> , 3)
in	<i>rot1,rot2</i>	Rotation vectors for each encounter (<i>n</i> , 3)
in	<i>solute_crds</i>	Coordinates of solute atoms (<i>nb_atoms</i> , 3)

10.2.1.4 merge_sorted_segments()

```

subroutine mod_array::merge_sorted_segments (
    real(kind=8), dimension(:), intent(inout) arr,
    type(type_assoc_file), intent(inout) complexes,
    real(kind=8), dimension(:), intent(inout) tmp_arr,
    character(len=217), dimension(:), intent(inout) tmp_comp,
    integer, intent(in) left,
    integer, intent(in) mid,
    integer, intent(in) right )

```

Merge two adjacent sorted segments of arrays in place.

Merges `arr(left:mid)` and `arr(mid+1:right)` into a single sorted segment `arr(left:right)`, re-ordering complexes and transform fields in the same way.

Parameters

in, out	<i>arr</i>	Array of values being sorted
in, out	<i>complexes</i>	Association object reordered alongside <i>arr</i>
out	<i>tmp_arr</i>	Temporary buffer for values (at least size <i>right</i>)
out	<i>tmp_comp</i>	Temporary buffer for companion (at least size <i>right</i>)
in	<i>left</i>	Start index of the first segment
in	<i>mid</i>	End index of the first segment
in	<i>right</i>	End index of the second segment

10.2.1.5 read_array()

```

subroutine mod_array::read_array (
    real(kind=8), dimension(:), intent(out), allocatable array,
    integer, intent(inout) n,
    character*128, intent(in) filename )

```

Read a numeric array from a formatted text file.

Parses a single-row formatted array file produced by `write_array`.

Parameters

out	<i>array</i>	Output array read from file (allocated)
in, out	<i>n</i>	Number of encounters (adjusted if file is smaller)
in	<i>filename</i>	Input filename

10.2.1.6 write_array()

```
subroutine mod_array::write_array (
    real(kind=8), dimension(:), intent(in) array,
    character*128, intent(in) filename )
```

Write a numeric array to a formatted text file.

Writes *array* as a single formatted line using F10.4 floats.

Parameters

in	<i>array</i>	Input array to write
in	<i>filename</i>	Output filename

10.3 mod_assoc Module Reference**Data Types**

- type [type_assoc_file](#)
Data type for storing binned data (eg.

Functions/Subroutines

- subroutine [allocate_assoc_object](#) (this, nlines)
Allocate storage for an association/complexes object.
- integer function [size_assoc](#) (assoc_file)
Count encounter records in an association/complexes file.
- subroutine [fill_assoc_object](#) (this, assoc_file)
Read an association/complexes file and populate [type_assoc_file](#).
- subroutine [write_complexes](#) (complexes, nb_encounters, complexes_filename)

10.3.1 Function/Subroutine Documentation

10.3.1.1 allocate_assoc_object()

```
subroutine mod_assoc::allocate_assoc_object (
    type(type_assoc_file), intent(inout) this,
    integer, intent(in) nlines )
```

Allocate storage for an association/complexes object.

Allocates the arrays inside *this* to hold *nlines* encounter records and initializes numeric arrays to zero.

Parameters

in, out	<i>this</i>	Instance of type_assoc_file to initialize
in	<i>nlines</i>	Number of encounter lines to allocate

10.3.1.2 fill_assoc_object()

```
subroutine mod_assoc::fill_assoc_object (
    type(type_assoc_file), intent(inout) this,
    integer, intent(in) assoc_file )
```

Read an association/complexes file and populate [type_assoc_file](#).

Reads header lines from the open unit *assoc_file* and extracts the two reference centers (*xc1*, *xc2*) from header lines 3 and 4. Subsequent non-header lines are stored into *thislines* and the translation and rotation vectors are parsed into *thistrans_vector*, *thisrot1* and *thisrot2*.

Parameters

in, out	<i>this</i>	Allocated type_assoc_file to populate
in	<i>assoc_file</i>	Fortran unit number of an already-open file

10.3.1.3 size_assoc()

```
integer function mod_assoc::size_assoc (
    integer assoc_file )
```

Count encounter records in an association/complexes file.

Scans *assoc_file* (a Fortran unit number) and counts the number of non-comment lines. Lines starting with # are treated as comments and ignored. Returns the number of encounter records.

Parameters

in	<i>assoc_file</i>	Fortran unit number of an already-open file
----	-------------------	---

Returns

nlines Number of encounter records found

10.3.1.4 write_complexes()

```
subroutine mod_assoc::write_complexes (
    type(type_assoc_file), intent(in) complexes,
    integer, intent(in) nb_encounters,
    character(len=*), intent(in) complexes_filename )
```

10.4 mod_clust_algorithm Module Reference

Functions/Subroutines

- subroutine [linkage_clustering_from_matrix](#) (matrix, n, linkage_type, output_name, complexes)
Perform hierarchical clustering on a distance matrix.
- subroutine [linkage_clustering_from_array](#) (array, n, linkage_type, output_name, complexes)
Perform hierarchical clustering on a distance array.
- subroutine [sort_complexes](#) (tot_encounters, cluster_indexes, cluster_count, active_clusters, representative_indexes, cluster_average, cluster_sd, array)
Sort clusters and associated arrays by a given key array.
- subroutine [write_cluster_elements](#) (tot_encounters, cluster_parent, cluster_count, active_clusters, output_name)
Write cluster membership lists to a text file.
- subroutine [write_clust_info](#) (tot_encounters, representative_indexes, active_clusters, clust_count, cluster_average, cluster_sd, output_name)
Write summary information about clusters to a text file.
- subroutine [write_cluster_complexes](#) (tot_encounters, cluster_parent, cluster_count, active_clusters, output_name, complexes)
Write complexes contained in each cluster to individual files.

10.4.1 Function/Subroutine Documentation

10.4.1.1 linkage_clustering_from_array()

```
subroutine mod_clust_algorithm::linkage_clustering_from_array (
    real (kind=8), dimension(:), intent(inout), allocatable array,
    integer, intent(in) n,
    character(len=*), intent(in) linkage_type,
    character*128, intent(in) output_name,
    type(type_assoc_file) complexes )
```

Perform hierarchical clustering on a distance array.

This routine performs hierarchical clustering using minimum, maximum, or mean linkage on the input `array` (shape `(n)`) of pairwise distances or similarity scores. The routine mutates `array` during merging and terminates merging when clusters are farther apart than an internally computed threshold. Results are written to files named using `output_name` and complexes information is optionally recorded via the `complexes` structure.

Parameters

in, out	<i>array</i>	Array of per-encounter values (n) This array is updated during clustering
in	<i>n</i>	Number of elements / encounters
in	<i>linkage_type</i>	Type of linkage: 'min', 'max', or 'mean'
in	<i>output_name</i>	Base name for output files
in	<i>complexes</i>	Structure containing complex text lines/headers

10.4.1.2 linkage_clustering_from_matrix()

```
subroutine mod_clust_algorithm::linkage_clustering_from_matrix (
    real (kind=8), dimension(:, :), intent(inout), allocatable matrix,
    integer, intent(in) n,
    character(len=*), intent(in) linkage_type,
    character*128, intent(in) output_name,
    type(type_assoc_file) complexes )
```

Perform hierarchical clustering on a distance matrix.

This routine performs hierarchical clustering using minimum, maximum, or mean linkage on the input *matrix* (shape (n, n)) of pairwise distances or similarity scores. The routine mutates *matrix* during merging and terminates merging when clusters are farther apart than an internally computed threshold. Results are written to files named using *output_name* and *complexes* information is optionally recorded via the *complexes* structure.

Parameters

in, out	<i>matrix</i>	Pairwise distance/similarity matrix (n, n) This matrix is updated during clustering
in	<i>n</i>	Number of elements / encounters
in	<i>linkage_type</i>	Type of linkage: 'min', 'max', or 'mean'
in	<i>output_name</i>	Base name for output files
in	<i>complexes</i>	Structure containing complex text lines/headers

10.4.1.3 sort_complexes()

```
subroutine mod_clust_algorithm::sort_complexes (
    integer, intent(in) tot_encounters,
    integer, dimension(:, :), intent(inout) cluster_indexes,
    integer, dimension(:, :), intent(inout) cluster_count,
    logical, dimension(:, :), intent(inout) active_clusters,
    integer, dimension(:, :), intent(inout) representative_indexes,
    real (kind=8), dimension(:, :), intent(inout) cluster_average,
    real (kind=8), dimension(:, :), intent(inout) cluster_sd,
    real (kind=8), dimension(:, :), intent(inout) array )
```

Sort clusters and associated arrays by a given key array.

Reorders *cluster_indexes* and all supplied per-cluster arrays so that they are sorted by ascending values in *array*. This is a stable reordering helper used prior to writing results.

Parameters

in	<i>tot_encounters</i>	Number of clusters/encounters
in, out	<i>cluster_indexes</i>	Matrix storing cluster member indexes
in, out	<i>cluster_count</i>	Number of members in each cluster
in, out	<i>active_clusters</i>	Logical flags marking active clusters
in, out	<i>representative_indexes</i>	Representative index per cluster
in, out	<i>cluster_average</i>	Per-cluster average values
in, out	<i>cluster_sd</i>	Per-cluster standard deviation
in, out	<i>array</i>	Key array used to sort clusters

10.4.1.4 write_clust_info()

```

subroutine mod_clust_algorithm::write_clust_info (
    integer, intent(in) tot_encounters,
    integer, dimension(:), intent(in) representative_indexes,
    logical, dimension(:), intent(in) active_clusters,
    integer, dimension(tot_encounters), intent(in) clust_count,
    real (kind=8), dimension(tot_encounters), intent(in) cluster_average,
    real (kind=8), dimension(tot_encounters), intent(in) cluster_sd,
    character*128, intent(in) output_name )

```

Write summary information about clusters to a text file.

Produces a summary file <output_name>_clust_info.txt listing cluster population, representative index, average and standard deviation for each active cluster.

Parameters

in	<i>tot_encounters</i>	Total number of encounters
in	<i>representative_indexes</i>	Representative index per cluster
in	<i>active_clusters</i>	Logical mask indicating active clusters
in	<i>clust_count</i>	Array with population of each cluster
in	<i>cluster_average</i>	Per-cluster average values
in	<i>cluster_sd</i>	Per-cluster standard deviations
in	<i>output_name</i>	Base filename for outputs

10.4.1.5 write_cluster_complexes()

```

subroutine mod_clust_algorithm::write_cluster_complexes (
    integer, intent(in) tot_encounters,
    integer, dimension(:), intent(in) cluster_parent,
    integer, dimension(:), intent(in) cluster_count,
    logical, dimension(:), intent(in) active_clusters,
    character*128, intent(in) output_name,
    type(type_assoc_file) complexes )

```

Write complexes contained in each cluster to individual files.

For each active cluster this routine creates a file <output_name>_clusterXXXX_complexes containing the header lines from `complexeshead` and the encounter lines for the cluster members taken from `complexeslines`.

Parameters

in	<i>tot_encounters</i>	Number of clusters/encounters
in	<i>cluster_parent</i>	Parent cluster for each encounter
in	<i>cluster_count</i>	Number of members per cluster
in	<i>active_clusters</i>	Logical mask indicating active clusters
in	<i>output_name</i>	Base filename for output files
in	<i>complexes</i>	Structure with header and lines to write

10.4.1.6 write_cluster_elements()

```
subroutine mod_clust_algorithm::write_cluster_elements (
    integer, intent(in) tot_encounters,
    integer, dimension(:), intent(in) cluster_parent,
    integer, dimension(:), intent(in) cluster_count,
    logical, dimension(:), intent(in) active_clusters,
    character*128, intent(in) output_name )
```

Write cluster membership lists to a text file.

Outputs a file named <output_name>_clusters.txt containing the members of each active cluster in a matrix-like format.

Parameters

in	<i>tot_encounters</i>	Number of possible clusters/encounters
in	<i>cluster_parent</i>	Parent cluster for each encounter
in	<i>cluster_count</i>	Number of members per cluster
in	<i>active_clusters</i>	Logical mask indicating active clusters
in	<i>output_name</i>	Base filename for output

10.5 mod_matrix Module Reference

Functions/Subroutines

- subroutine [matrix_rmsd](#) (matrix, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, coord)
Build RMSD-based pairwise matrix between transformed coordinates.
- subroutine [write_matrix](#) (matrix, filename)
Write a numeric matrix to a formatted text file.
- subroutine [read_matrix](#) (matrix, n, filename)
Read a square numeric matrix from a formatted text file.

10.5.1 Function/Subroutine Documentation

10.5.1.1 `matrix_rmsd()`

```
subroutine mod_matrix::matrix_rmsd (
    real(kind=8), dimension(n, n), intent(out) matrix,
    integer, intent(in) n,
    integer, intent(in) nb_atoms,
    real(kind=8), dimension(3), intent(in) xc1,
    real(kind=8), dimension(3), intent(in) xc2,
    real(kind=8), dimension(:, :), intent(in) trans_vector,
    real(kind=8), dimension(:, :), intent(in) rot1,
    real(kind=8), dimension(:, :), intent(in) rot2,
    real(kind=8), dimension(:, :), intent(in) coord )
```

Build RMSD-based pairwise matrix between transformed coordinates.

Applies transformation to the coordinate set for each encounter and computes pairwise RMSD values between transformed coordinate sets.

Parameters

out	<i>matrix</i>	Output (n,n) matrix of pairwise RMSD values
in	<i>n</i>	Number of encounters
in	<i>nb_atoms</i>	Number of atoms in solute2 to transform
in	<i>xc1,xc2</i>	Solute centers for translation
in	<i>trans_vector</i>	Transform translation vectors (n,3)
in	<i>rot1,rot2</i>	Transform rotation arrays (n,3)
in	<i>coord</i>	Coordinates to be transformed

10.5.1.2 `read_matrix()`

```
subroutine mod_matrix::read_matrix (
    real(kind=8), dimension(:, :), intent(out), allocatable matrix,
    integer, intent(inout) n,
    character*128, intent(in) filename )
```

Read a square numeric matrix from a formatted text file.

Reads the file line-by-line to determine matrix size and parses rows with the expected F10.4 formatting.

Parameters

out	<i>matrix</i>	Output matrix read from file (allocated)
in, out	<i>n</i>	Number of encounters (adjusted if file is smaller)
in	<i>filename</i>	Input filename

10.5.1.3 write_matrix()

```
subroutine mod_matrix::write_matrix (
    real(kind=8), dimension(:, :), intent(in) matrix,
    character*128, intent(in) filename )
```

Write a numeric matrix to a formatted text file.

Writes matrix with each row as F10.4 formatted floats. Output filename is provided by filename.

Parameters

in	<i>matrix</i>	Input matrix to write
in	<i>filename</i>	Output filename

10.6 mod_pdb Module Reference

Data Types

- type [type_pdb_atom](#)
- type [type_pdb_chain](#)
- type [type_pdb_file](#)
Data type for storing binned data (eg.
- type [type_pdb_residue](#)

Functions/Subroutines

- subroutine [allocate_pdb_object](#) (this, pdb_file, natoms, nresidues, nchains)
Allocate and initialize a [type_pdb_file](#) structure from a PDB file.
- integer function [count_atoms](#) (pdb_file)
Count ATOM/HETATM records in a PDB file.
- integer function [count_residues](#) (pdb_file)
Count residues in a PDB file.
- integer function [count_chains](#) (pdb_file)
Count chain separators (TER records) in a PDB file.
- subroutine [fill_pdb_object](#) (this, pdb_file)
Parse PDB records and populate the [type_pdb_file](#) structure.

10.6.1 Function/Subroutine Documentation

10.6.1.1 allocate_pdb_object()

```
subroutine mod_pdb::allocate_pdb_object (
    type(type_pdb_file), intent(inout) this,
    integer, intent(in) pdb_file,
    integer, intent(in) natoms,
    integer, intent(in) nresidues,
    integer, intent(in) nchains )
```

Allocate and initialize a `type_pdb_file` structure from a PDB file.

This routine allocates the storage for `thisatoms`, `thisresidues` and `thischains` using the supplied counts and performs a preliminary scan of the open Fortran unit `pdb_file` to determine per-chain and per-residue allocation sizes for nested arrays.

Parameters

in, out	<i>this</i>	<code>type_pdb_file</code> instance to allocate and initialize
in	<i>pdb_file</i>	Fortran unit number of an already-open PDB file
in	<i>natoms</i>	Total number of ATOM/HETATM records expected
in	<i>nresidues</i>	Number of residues expected
in	<i>nchains</i>	Number of chains expected

10.6.1.2 count_atoms()

```
integer function mod_pdb::count_atoms (
    integer pdb_file )
```

Count ATOM/HETATM records in a PDB file.

Scans the open Fortran file unit `pdb_file` and returns the number of lines starting with ATOM or HETATM.

Parameters

in	<i>pdb_file</i>	Fortran unit number of an already-open PDB file
----	-----------------	---

Returns

`natoms` Number of ATOM/HETATM records found

10.6.1.3 count_chains()

```
integer function mod_pdb::count_chains (
    integer pdb_file )
```

Count chain separators (TER records) in a PDB file.

Scans the open Fortran file unit `pdb_file` and counts the number of TER records, which are treated as chain separators. Returns the number of chains found.

Parameters

in	<i>pdb_file</i>	Fortran unit number of an already-open PDB file
----	-----------------	---

Returns

nchains Number of chains (TER records) found

10.6.1.4 count_residues()

```
integer function mod_pdb::count_residues (
    integer pdb_file )
```

Count residues in a PDB file.

Scans the open Fortran file unit *pdb_file* and counts residue changes by comparing residue identifiers in ATOM/HETATM records. Returns the number of distinct residues found.

Parameters

in	<i>pdb_file</i>	Fortran unit number of an already-open PDB file
----	-----------------	---

Returns

nresidues Number of residues found

10.6.1.5 fill_pdb_object()

```
subroutine mod_pdb::fill_pdb_object (
    type(type_pdb_file), intent(inout) this,
    integer, intent(in) pdb_file )
```

Parse PDB records and populate the *type_pdb_file* structure.

Reads ATOM/HETATM records from the already-open Fortran unit *pdb_file*, fills *this* atoms with atom records and populates *this*residues and *this*chains so that residues contain their per-residue atoms arrays and chains contain their residues and atoms. The routine expects *this* to have been allocated with the correct sizes (see *allocate_pdb_object*).

Parameters

in, out	<i>this</i>	Pre-allocated <i>type_pdb_file</i> to populate
in	<i>pdb_file</i>	Fortran unit number of an already-open PDB file

10.7 mod_threshold Module Reference

Functions/Subroutines

- subroutine [array_z_coord](#) (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute_crds)
Compute the Z-coordinate (along transformed z-axis) for each encounter.
- subroutine [array_atoms_dist](#) (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute1_crds, solute2_crds)
Compute minimum inter-atomic distance between two solutes for each encounter.
- subroutine [array_angle](#) (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, point1a, point1b, point2a, point2b, dimensions)
Compute angle between two vectors (defined by atom groups) per encounter.
- subroutine [sort_array](#) (arr, encounter_indexes)
Sort an array in ascending order and reorder encounter indexes accordingly.

10.7.1 Function/Subroutine Documentation

10.7.1.1 array_angle()

```
subroutine mod_threshold::array_angle (
    real(kind=8), dimension(n), intent(out) array,
    integer, intent(in) n,
    integer, intent(in) nb_atoms,
    real(kind=8), dimension(3), intent(in) xc1,
    real(kind=8), dimension(3), intent(in) xc2,
    real(kind=8), dimension(:, :), intent(in) trans_vector,
    real(kind=8), dimension(:, :), intent(in) rot1,
    real(kind=8), dimension(:, :), intent(in) rot2,
    real(kind=8), dimension(:), intent(in) point1a,
    real(kind=8), dimension(:), intent(in) point1b,
    real(kind=8), dimension(:), intent(in) point2a,
    real(kind=8), dimension(:), intent(in) point2b,
    integer, intent(in) dimensions )
```

Compute angle between two vectors (defined by atom groups) per encounter.

Transforms the pair of points from `solute2` for each encounter and computes the angle between the vector defined by `point1b-point1a` and the vector defined by the transformed `point2b-point2a`.

Parameters

out	<i>array</i>	Output array of length <i>n</i> with angles in degrees
in	<i>n</i>	Number of encounters
in	<i>nb_atoms</i>	Number of atoms used for transformation
in	<i>xc1,xc2</i>	Centers used for transform
in	<i>trans_vector</i>	Translation vectors for each encounter (<i>n</i> , 3)
in	<i>rot1,rot2</i>	Rotation vectors for each encounter (<i>n</i> , 3)
in	<i>point1a</i>	First point defining reference vector
in	<i>point1b</i>	Second point defining reference vector
in	<i>point2a</i>	First point of vector to be transformed
in	<i>point2b</i>	Second point of vector to be transformed
in	<i>dimensions</i>	Use 2 or 3 to select projection

10.7.1.2 array_atoms_dist()

```

subroutine mod_threshold::array_atoms_dist (
    real(kind=8), dimension(n), intent(out) array,
    integer, intent(in) n,
    integer, intent(in) nb_atoms,
    real(kind=8), dimension(3), intent(in) xc1,
    real(kind=8), dimension(3), intent(in) xc2,
    real(kind=8), dimension(:, :), intent(in) trans_vector,
    real(kind=8), dimension(:, :), intent(in) rot1,
    real(kind=8), dimension(:, :), intent(in) rot2,
    real(kind=8), dimension(:, :), intent(in) solute1_crds,
    real(kind=8), dimension(:, :), intent(in) solute2_crds )

```

Compute minimum inter-atomic distance between two solutes for each encounter.

For each encounter, the second solute is transformed and the minimum distance between any atom of `solute1_crds` and the transformed `solute2_crds` is computed and stored in `array`.

Parameters

out	<i>array</i>	Output array of length <i>n</i> with minimum distances
in	<i>n</i>	Number of encounters
in	<i>nb_atoms</i>	Number of atoms in <code>solute2_crds</code>
in	<i>xc1,xc2</i>	Centers used for transform
in	<i>trans_vector</i>	Translation vectors for each encounter (<i>n</i> , 3)
in	<i>rot1,rot2</i>	Rotation vectors for each encounter (<i>n</i> , 3)
in	<i>solute1_crds</i>	Coordinates of solute1 atoms
in	<i>solute2_crds</i>	Coordinates of solute2 atoms

10.7.1.3 array_z_coord()

```

subroutine mod_threshold::array_z_coord (
    real(kind=8), dimension(n), intent(out) array,
    integer, intent(in) n,
    integer, intent(in) nb_atoms,
    real(kind=8), dimension(3), intent(in) xc1,
    real(kind=8), dimension(3), intent(in) xc2,
    real(kind=8), dimension(:, :), intent(in) trans_vector,
    real(kind=8), dimension(:, :), intent(in) rot1,
    real(kind=8), dimension(:, :), intent(in) rot2,
    real(kind=8), dimension(:, :), intent(in) solute_crds )

```

Compute the Z-coordinate (along transformed z-axis) for each encounter.

Recenters each encounter at `xc2`, applies rotation/translation using `rot1/rot2` and `trans_vector`, and extracts the Z coordinate of the first atom in `solute_crds` for every encounter into `array`.

Parameters

out	<i>array</i>	Output array of length <i>n</i> with z-coordinates
in	<i>n</i>	Number of encounters
in	<i>nb_atoms</i>	Number of atoms in <i>solute_crds</i>
in	<i>xc1,xc2</i>	Centers used for transform
in	<i>trans_vector</i>	Translation vectors for each encounter (<i>n</i> , 3)
in	<i>rot1,rot2</i>	Rotation vectors for each encounter (<i>n</i> , 3)
in	<i>solute_crds</i>	Coordinates of solute atoms (<i>nb_atoms</i> , 3)

10.7.1.4 sort_array()

```
subroutine mod_threshold::sort_array (
    real (kind=8), dimension(:), intent(inout) arr,
    integer, dimension(:), intent(inout) encounter_indexes )
```

Sort an array in ascending order and reorder encounter indexes accordingly.

Simple selection sort used to reorder *arr* and the parallel *encounter_indexes* array. Intended for small to moderate sizes.

Parameters

in, out	<i>arr</i>	Array to sort (modified in-place)
in, out	<i>encounter_indexes</i>	Parallel index array reordered to match <i>arr</i>

10.8 read_input Module Reference

Functions/Subroutines

- subroutine [read_pdb](#) (*pdb*, *filename*, *tot_atoms*, *tot_residues*, *tot_chains*)
Read a PDB file into a type_pdb_file object.
- subroutine [read_assoc](#) (*assoc*, *filename*, *tot_encounters*)
Read an association/complexes file into a type_assoc_file object.

10.8.1 Function/Subroutine Documentation

10.8.1.1 read_assoc()

```
subroutine read_input::read_assoc (
    type ( type_assoc_file ) assoc,
    character*128, intent(in) filename,
    integer, intent(out) tot_encounters )
```

Read an association/complexes file into a `type_assoc_file` object.

Checks the file exists, opens it and uses `size_assoc`, `allocate_assoc_object` and `fill_assoc_↵` object to construct the `assoc` structure.

Parameters

out	<i>assoc</i>	Allocated and filled assoc object
in	<i>filename</i>	Path to complexes file
out	<i>tot_encounters</i>	Number of encounter records read

10.8.1.2 read_pdb()

```
subroutine read_input::read_pdb (
    type ( type_pdb_file ) pdb,
    character*128, intent(in) filename,
    integer, intent(out) tot_atoms,
    integer, intent(out) tot_residues,
    integer, intent(out) tot_chains )
```

Read a PDB file into a `type_pdb_file` object.

Validates the existence of `filename`, opens it, determines the counts for atoms, residues and chains, allocates `pdb` and fills it using `allocate_pdb_object` and `fill_pdb_object`.

Parameters

out	<i>pdb</i>	Allocated and filled PDB object
in	<i>filename</i>	Path to PDB file
out	<i>tot_atoms</i>	Number of atoms found
out	<i>tot_residues</i>	Number of residues found
out	<i>tot_chains</i>	Number of chains found

Chapter 11

Data Type Documentation

11.1 `mod_assoc::type_assoc_file` Type Reference

Data type for storing binned data (eg.

Public Attributes

- integer `nlines`
Number of lines in assoc_file file.
- character(len=217), dimension(:), allocatable `lines`
- character(len=217), dimension(4) `head`
- real(kind=8), dimension(:, :), allocatable `trans_vector`
Translational vector.
- real(kind=8), dimension(:, :), allocatable `rot1`
rotX, rotY
- real(kind=8), dimension(:, :), allocatable `rot2`
- real(kind=8), dimension(3) `xc1`
center of mass of p2
- real(kind=8), dimension(3) `xc2`

11.1.1 Detailed Description

Data type for storing binned data (eg.

Radial distribution functions)

11.1.2 Member Data Documentation

11.1.2.1 head

```
character(len=217), dimension(4) mod_assoc::type_assoc_file::head
```

11.1.2.2 lines

```
character(len=217), dimension(:), allocatable mod_assoc::type_assoc_file::lines
```

11.1.2.3 nlines

```
integer mod_assoc::type_assoc_file::nlines
```

Number of lines in assoc_file file.

11.1.2.4 rot1

```
real (kind=8), dimension (:,:), allocatable mod_assoc::type_assoc_file::rot1
```

rotX, rotY

11.1.2.5 rot2

```
real (kind=8), dimension (:,:), allocatable mod_assoc::type_assoc_file::rot2
```

11.1.2.6 trans_vector

```
real (kind=8), dimension (:, :), allocatable mod_assoc::type_assoc_file::trans_vector
```

Translational vector.

11.1.2.7 xc1

```
real (kind=8), dimension (3) mod_assoc::type_assoc_file::xc1
```

center of mass of p2

11.1.2.8 xc2

```
real (kind=8), dimension (3) mod_assoc::type_assoc_file::xc2
```

The documentation for this type was generated from the following file:

- [mod_assoc.f90](#)

11.2 mod_pdb::type_pdb_atom Type Reference

Public Attributes

- integer [record](#)
Number of atoms in pdb file Flag for record type, 1 = ATOM, 2 = HETATM.
- real(kind=8), dimension(3) [coord](#)
Position, occupancy and temp factor - can be used to store other data eg in pqr.
- real(kind=8) [occ](#)
- real(kind=8) [beta](#)
- character(len=4) [name](#)
Atom name, res name and chain name.
- character(len=3) [resname](#)
- character(len=1) [chainid](#)
- integer [id](#)
Atom and residue numbers.
- integer [resid](#)

11.2.1 Member Data Documentation

11.2.1.1 beta

```
real (kind=8) mod_pdb::type_pdb_atom::beta
```

11.2.1.2 chainid

```
character (len=1) mod_pdb::type_pdb_atom::chainid
```

11.2.1.3 coord

```
real (kind=8), dimension (3) mod_pdb::type_pdb_atom::coord
```

Position, occupancy and temp factor - can be used to store other data eg in pqr.

11.2.1.4 id

```
integer mod_pdb::type_pdb_atom::id
```

Atom and residue numbers.

11.2.1.5 name

```
character (len=4) mod_pdb::type_pdb_atom::name
```

Atom name, res name and chain name.

11.2.1.6 occ

```
real (kind=8) mod_pdb::type_pdb_atom::occ
```

11.2.1.7 record

```
integer mod_pdb::type_pdb_atom::record
```

Number of atoms in pdb file Flag for record type, 1 = ATOM, 2 = HETATM.

11.2.1.8 resid

```
integer mod_pdb::type_pdb_atom::resid
```

11.2.1.9 resname

```
character (len=3) mod_pdb::type_pdb_atom::resname
```

The documentation for this type was generated from the following file:

- [mod_pdb.f90](#)

11.3 mod_pdb::type_pdb_chain Type Reference

Collaboration diagram for mod_pdb::type_pdb_chain:

11.4 mod_pdb::type_pdb_file Type Reference

Data type for storing binned data (eg.

Collaboration diagram for mod_pdb::type_pdb_file:

Public Attributes

- integer [natoms](#)
Number of atoms in pdb file.
- integer [nresidues](#)
- integer [nchains](#)
- type([type_pdb_atom](#)), dimension(:), allocatable [atoms](#)
Flag for record type, 1 = ATOM, 2 = HETATM.
- type([type_pdb_residue](#)), dimension(:), allocatable [residues](#)
- type([type_pdb_chain](#)), dimension(:), allocatable [chains](#)

11.4.1 Detailed Description

Data type for storing binned data (eg.

Radial distribution functions)

11.4.2 Member Data Documentation

11.4.2.1 atoms

```
type(type\_pdb\_atom), dimension (:), allocatable mod_pdb::type_pdb_file::atoms
```

Flag for record type, 1 = ATOM, 2 = HETATM.

11.4.2.2 chains

```
type(type\_pdb\_chain), dimension (:), allocatable mod_pdb::type_pdb_file::chains
```

11.4.2.3 natoms

```
integer mod_pdb::type_pdb_file::natoms
```

Number of atoms in pdb file.

11.4.2.4 nchains

```
integer mod_pdb::type_pdb_file::nchains
```

11.4.2.5 nresidues

```
integer mod_pdb::type_pdb_file::nresidues
```

11.4.2.6 residues

```
type(type_pdb_residue), dimension (:), allocatable mod_pdb::type_pdb_file::residues
```

The documentation for this type was generated from the following file:

- [mod_pdb.f90](#)

11.5 mod_pdb::type_pdb_residue Type Reference

Collaboration diagram for mod_pdb::type_pdb_residue:

Public Attributes

- integer [natoms](#)
Number of atoms in pdb file.
- character [chainid](#)
Chain id.
- character(len=3) [resname](#)
Residue name.
- integer [resid](#)
Residue number.
- type([type_pdb_atom](#)), dimension(:), allocatable [atoms](#)
Flag for record type, 1 = ATOM, 2 = HETATM.

11.5.1 Member Data Documentation

11.5.1.1 atoms

`type(type_pdb_atom), dimension (:), allocatable mod_pdb::type_pdb_residue::atoms`

Flag for record type, 1 = ATOM, 2 = HETATM.

11.5.1.2 chainid

`character mod_pdb::type_pdb_residue::chainid`

Chain id.

11.5.1.3 natoms

`integer mod_pdb::type_pdb_residue::natoms`

Number of atoms in pdb file.

11.5.1.4 resid

`integer mod_pdb::type_pdb_residue::resid`

Residue number.

11.5.1.5 resname

`character (len=3) mod_pdb::type_pdb_residue::resname`

Residue name.

The documentation for this type was generated from the following file:

- [mod_pdb.f90](#)

Chapter 12

File Documentation

12.1 `analyze_residues.f90` File Reference

Residue proximity analysis across transformed encounters.

Functions/Subroutines

- program [main](#)
- subroutine [print_help](#) ()

Print usage/help text for `analyze_residues`.

12.1.1 Detailed Description

Residue proximity analysis across transformed encounters.

[analyze_residues.f90](#) - Find residues close to the surface across encounters.

Usage: `./analyze_residues -help`

Reads a PDB and complexes file, transforms residue centers, and counts how many encounters bring residues within a threshold. Results: `closest_residues.txt`

Author

Abraham Muñoz-Chicharro

Version

1.0

Date

2026-04-05

12.1.2 Function/Subroutine Documentation

12.1.2.1 `main()`

program `main`

12.1.2.2 `print_help()`

subroutine `main::print_help`

Print usage/help text for `analyze_residues`.

Shows required command-line options and examples for running the residue analysis tool.

12.2 `analyze_residues.md` File Reference

12.3 `clust.f90` File Reference

Hierarchical clustering driver for encounter matrices/arrays.

Functions/Subroutines

- program `main`
- subroutine `parse_arguments` (`p_complexes_filename`, `p_complexes_bool`, `p_nb_encounters`, `p_nb_encounters_bool`, `p_datadist_filename`, `p_output_name`, `p_output_name_bool`, `p_linkage_type`, `p_help_bool`)
Parse command-line arguments for `clust`.
- subroutine `print_help` ()
Print usage and help information for the `clust` program.

12.3.1 Detailed Description

Hierarchical clustering driver for encounter matrices/arrays.

`clust.f90` - Hierarchical clustering for encounter matrices.

Usage: `./clust -help`

Reads a pairwise matrix (or array) and runs hierarchical clustering with user-specified linkage. Results written to `output_name` files.

Author

Abraham Muñiz-Chicharro

Version

1.0

Date

2026-04-05

12.3.2 Function/Subroutine Documentation

12.3.2.1 main()

```
program main
```

12.3.2.2 parse_arguments()

```
subroutine main::parse_arguments (
    character*128, intent(out) p_complexes_filename,
    logical, intent(out) p_complexes_bool,
    integer, intent(out) p_nb_encounters,
    logical, intent(out) p_nb_encounters_bool,
    character*128, intent(out) p_datadist_filename,
    character*128, intent(out) p_output_name,
    logical, intent(out) p_output_name_bool,
    character*128, intent(out) p_linkage_type,
    logical, intent(out) p_help_bool )
```

Parse command-line arguments for `clust`.

Parses all CLI flags and fills program configuration values.

Parameters

out	<i>p_complexes_filename</i>	Value parsed from <code>-complexes</code>
out	<i>p_complexes_bool</i>	Whether <code>-complexes</code> was provided
out	<i>p_nb_encounters</i>	Value parsed from <code>-nb_encounters</code>
out	<i>p_nb_encounters_bool</i>	Whether <code>-nb_encounters</code> was provided
out	<i>p_datadist_filename</i>	Value parsed from <code>-input</code>
out	<i>p_output_name</i>	Value parsed from <code>-output_name</code>
out	<i>p_output_name_bool</i>	Whether <code>-output_name</code> was provided
out	<i>p_linkage_type</i>	Value parsed from <code>-linkage</code> (default mean)
out	<i>p_help_bool</i>	Whether <code>-help</code> was provided

12.3.2.3 print_help()

```
subroutine main::print_help
```

Print usage and help information for the `clust` program.

Describes command-line options and provides an example invocation.

12.4 clust.md File Reference

12.5 clust_all.f90 File Reference

Build matrices/arrays and perform clustering in memory (no disk I/O). Usage: ./clust_all -help.

Functions/Subroutines

- program [clust_all](#)
- subroutine [parse_arguments](#) (p_pdb1_filename, p_pdb1_bool, p_pdb2_filename, p_pdb2_bool, p_↔ complexes_filename, p_complexes_bool, p_nb_encounters, p_nb_encounters_bool, p_data_type, p_↔ datatype_bool, p_datadist_filename, p_datadist_filename_bool, p_linkage_type, p_output_name, p_arr_↔ atoms1a, p_arr_atoms1a_bool, p_arr_atoms1b, p_arr_atoms1b_bool, p_arr_atoms2a, p_arr_atoms2a_bool, p_arr_atoms2b, p_arr_atoms2b_bool, p_help_bool)
Parse command-line arguments for clust_all.
- subroutine [print_help](#) (help_option)
Print usage/help text for clust_all.
- subroutine [read_atoms_coord](#) (atoms_array, atoms_coord, tot_atoms, pdb, pdb_filename)
Read coordinates for a specified list of atoms (names or indices).

12.5.1 Detailed Description

Build matrices/arrays and perform clustering in memory (no disk I/O). Usage: ./clust_all -help.

This driver combines two traditionally separate phases: (1) metric generation from encounter transforms and atom selections, and (2) hierarchical clustering on the generated metric.

Compared with split workflows (`make_data` then `clust`), this program keeps intermediate numeric data in memory to reduce temporary files and simplify large-batch runs.

Author

Abraham Muñiz-Chicharro

Version

1.0

Date

2026-04-05

12.5.2 Function/Subroutine Documentation

12.5.2.1 clust_all()

```
program clust_all
```

12.5.2.2 parse_arguments()

```
subroutine clust_all::parse_arguments (
    character*128, intent(out) p_pdb1_filename,
    logical, intent(out) p_pdb1_bool,
    character*128, intent(out) p_pdb2_filename,
    logical, intent(out) p_pdb2_bool,
    character*128, intent(out) p_complexes_filename,
    logical, intent(out) p_complexes_bool,
    integer, intent(out) p_nb_encounters,
    logical, intent(out) p_nb_encounters_bool,
    character*128, intent(out) p_data_type,
    logical, intent(out) p_datatype_bool,
    character*128, intent(out) p_datadist_filename,
    logical, intent(out) p_datadist_filename_bool,
    character*128, intent(out) p_linkage_type,
    character*128, intent(out) p_output_name,
    character*128, dimension(:), intent(out), allocatable p_arr_atoms1a,
    logical, intent(out) p_arr_atoms1a_bool,
    character*128, dimension(:), intent(out), allocatable p_arr_atoms1b,
    logical, intent(out) p_arr_atoms1b_bool,
    character*128, dimension(:), intent(out), allocatable p_arr_atoms2a,
    logical, intent(out) p_arr_atoms2a_bool,
    character*128, dimension(:), intent(out), allocatable p_arr_atoms2b,
    logical, intent(out) p_arr_atoms2b_bool,
    logical, intent(out) p_help_bool )
```

Parse command-line arguments for clust_all.

Parses all CLI flags and fills program configuration values. Atom-group flags consume multiple tokens until the next flag.

Parameters

out	<i>p_pdb1_filename</i>	Value parsed from -pdb1
out	<i>p_pdb1_bool</i>	Whether -pdb1 was provided
out	<i>p_pdb2_filename</i>	Value parsed from -pdb2
out	<i>p_pdb2_bool</i>	Whether -pdb2 was provided
out	<i>p_complexes_filename</i>	Value parsed from -complexes
out	<i>p_complexes_bool</i>	Whether -complexes was provided
out	<i>p_nb_encounters</i>	Value parsed from -nb_encounters
out	<i>p_nb_encounters_bool</i>	Whether -nb_encounters was provided
out	<i>p_data_type</i>	Value parsed from -data_type
out	<i>p_datatype_bool</i>	Whether -data_type was provided
out	<i>p_datadist_filename</i>	Value parsed from -datadist
out	<i>p_datadist_filename_bool</i>	Whether -datadist was provided
out	<i>p_linkage_type</i>	Value parsed from -linkage (default min)
out	<i>p_output_name</i>	Value parsed from -output_name (default clust_all_output)

Parameters

out	<i>p_arr_atoms1a</i>	Atom list parsed from <code>-atoms1</code> or <code>-atoms1a</code>
out	<i>p_arr_atoms1a_bool</i>	Whether <code>-atoms1</code> or <code>-atoms1a</code> was provided
out	<i>p_arr_atoms1b</i>	Atom list parsed from <code>-atoms1b</code>
out	<i>p_arr_atoms1b_bool</i>	Whether <code>-atoms1b</code> was provided
out	<i>p_arr_atoms2a</i>	Atom list parsed from <code>-atoms2</code> or <code>-atoms2a</code>
out	<i>p_arr_atoms2a_bool</i>	Whether <code>-atoms2</code> or <code>-atoms2a</code> was provided
out	<i>p_arr_atoms2b</i>	Atom list parsed from <code>-atoms2b</code>
out	<i>p_arr_atoms2b_bool</i>	Whether <code>-atoms2b</code> was provided
out	<i>p_help_bool</i>	Whether <code>-help</code> was provided

12.5.2.3 print_help()

```
subroutine clust_all::print_help (
    character(len=*), intent(in) help_option )
```

Print usage/help text for `clust_all`.

`help_option` controls whether general usage or mode-specific requirements are printed.

12.5.2.4 read_atoms_coord()

```
subroutine clust_all::read_atoms_coord (
    character(len=*), dimension(:), intent(in) atoms_array,
    real(kind=8), dimension(:, :), intent(out), allocatable atoms_coord,
    integer, intent(in) tot_atoms,
    type ( type_pdb_file ), intent(in) pdb,
    character(len=*), intent(in) pdb_filename )
```

Read coordinates for a specified list of atoms (names or indices).

Accepts either atom indices or atom names. If atom names are provided, all matching atoms in the PDB are returned (in order).

Parameters

in	<i>atoms_array</i>	Array of atom name strings or index strings
out	<i>atoms_coord</i>	Allocated array with atom coordinates (n,3)
in	<i>tot_atoms</i>	Total number of atoms in <code>pdb</code>
in	<i>pdb</i>	PDB structure holding atom coordinates
in	<i>pdb_filename</i>	Reference filename for warnings

12.6 clust_all.md File Reference

12.7 make_data.f90 File Reference

Build encounter-based matrix/array data from structural inputs.

Functions/Subroutines

- program [main](#)
- subroutine [parse_arguments](#) (p_pdb1_filename, p_pdb1_bool, p_pdb2_filename, p_pdb2_bool, p_↵ complexes_filename, p_complexes_bool, p_nb_encounters, p_nb_encounters_bool, p_datadist_filename, p_matrix_bool, p_data_type, p_datatype_bool, p_arr_atoms1a, p_arr_atoms1a_bool, p_arr_atoms1b, p_↵ arr_atoms1b_bool, p_arr_atoms2a, p_arr_atoms2a_bool, p_arr_atoms2b, p_arr_atoms2b_bool, p_help_↵ bool)
Parse command-line arguments for make_data.
- subroutine [print_help](#) (help_option)
Print usage/help text for make_data.
- subroutine [read_atoms_coord](#) (atoms_array, atoms_coord, tot_atoms, pdb, pdb_filename)
Read coordinates for a specified list of atoms (names or indices).

12.7.1 Detailed Description

Build encounter-based matrix/array data from structural inputs.

[make_data.f90](#) - Build encounter-based matrices from PDB and complex files.

Usage: ./make_data -help

Reads PDB(s) and association/complex files, generates matrices (rmsd, z_coord, atoms_dist, angle) and arrays.

Author

Abraham Muñiz-Chicharro

Version

1.0

Date

2026-04-05

12.7.2 Function/Subroutine Documentation

12.7.2.1 main()

program main

12.7.2.2 parse_arguments()

```
subroutine main::parse_arguments (
    character*128, intent(out) p_pdb1_filename,
    logical, intent(out) p_pdb1_bool,
    character*128, intent(out) p_pdb2_filename,
    logical, intent(out) p_pdb2_bool,
    character*128, intent(out) p_complexes_filename,
    logical, intent(out) p_complexes_bool,
    integer, intent(out) p_nb_encounters,
    logical, intent(out) p_nb_encounters_bool,
    character*128, intent(out) p_datadist_filename,
    logical, intent(out) p_matrix_bool,
    character*128, intent(out) p_data_type,
    logical, intent(out) p_datatype_bool,
    character*128, dimension(:), intent(out), allocatable p_arr_atoms1a,
    logical, intent(out) p_arr_atoms1a_bool,
    character*128, dimension(:), intent(out), allocatable p_arr_atoms1b,
    logical, intent(out) p_arr_atoms1b_bool,
    character*128, dimension(:), intent(out), allocatable p_arr_atoms2a,
    logical, intent(out) p_arr_atoms2a_bool,
    character*128, dimension(:), intent(out), allocatable p_arr_atoms2b,
    logical, intent(out) p_arr_atoms2b_bool,
    logical, intent(out) p_help_bool )
```

Parse command-line arguments for make_data.

Parses all CLI flags and fills program configuration values. Atom-group flags consume multiple tokens until the next flag.

Parameters

out	<i>p_pdb1_filename</i>	Value parsed from -pdb1
out	<i>p_pdb1_bool</i>	Whether -pdb1 was provided
out	<i>p_pdb2_filename</i>	Value parsed from -pdb2
out	<i>p_pdb2_bool</i>	Whether -pdb2 was provided
out	<i>p_complexes_filename</i>	Value parsed from -complexes
out	<i>p_complexes_bool</i>	Whether -complexes was provided
out	<i>p_nb_encounters</i>	Value parsed from -nb_encounters
out	<i>p_nb_encounters_bool</i>	Whether -nb_encounters was provided
out	<i>p_datadist_filename</i>	Value parsed from -input
out	<i>p_matrix_bool</i>	Whether -input was provided
out	<i>p_data_type</i>	Value parsed from -data_type
out	<i>p_datatype_bool</i>	Whether -data_type was provided
out	<i>p_arr_atoms1a</i>	Atom list parsed from -atoms1 or -atoms1a
out	<i>p_arr_atoms1a_bool</i>	Whether -atoms1 or -atoms1a was provided
out	<i>p_arr_atoms1b</i>	Atom list parsed from -atoms1b

Parameters

out	<i>p_arr_atoms1b_bool</i>	Whether <code>-atoms1b</code> was provided
out	<i>p_arr_atoms2a</i>	Atom list parsed from <code>-atoms2</code> or <code>-atoms2a</code>
out	<i>p_arr_atoms2a_bool</i>	Whether <code>-atoms2</code> or <code>-atoms2a</code> was provided
out	<i>p_arr_atoms2b</i>	Atom list parsed from <code>-atoms2b</code>
out	<i>p_arr_atoms2b_bool</i>	Whether <code>-atoms2b</code> was provided
out	<i>p_help_bool</i>	Whether <code>-help</code> was provided

12.7.2.3 print_help()

```
subroutine main::print_help (
    character(len=*), intent(in) help_option )
```

Print usage/help text for `make_data`.

Provides short usage examples for the main program and for each supported matrix type.

12.7.2.4 read_atoms_coord()

```
subroutine main::read_atoms_coord (
    character(len=*), dimension(:), intent(in) atoms_array,
    real(kind=8), dimension(:, :), intent(out), allocatable atoms_coord,
    integer, intent(in) tot_atoms,
    type ( type_pdb_file ), intent(in) pdb,
    character(len=*), intent(in) pdb_filename )
```

Read coordinates for a specified list of atoms (names or indices).

Accepts either atom indices or atom names. If atom names are provided, all matching atoms in the PDB are returned (in order).

Parameters

in	<i>atoms_array</i>	Array of atom name strings or index strings
out	<i>atoms_coord</i>	Allocated array with atom coordinates (n,3)
in	<i>tot_atoms</i>	Total number of atoms in <code>pdb</code>
in	<i>pdb</i>	PDB structure holding atom coordinates
in	<i>pdb_filename</i>	Reference filename for warnings

12.8 make_data.md File Reference

12.9 maths.f90 File Reference

Mathematical and geometric utilities for clustering workflows.

Modules

- module [maths](#)

Functions/Subroutines

- subroutine [maths::cross](#) (v1, v2, v3)
Compute the cross product of two 3D vectors.
- subroutine [maths::update_complex](#) (xc1, xc2, trans, rot_vx, rot_vy, nb_atoms, coords, new_coords)
Apply translation and rotation to a set of atomic coordinates.
- subroutine [maths::vectors_angle_3d](#) (vector1, vector2, theta_degrees)
Compute the angle in degrees between two 3D vectors.
- subroutine [maths::vectors_angle_2d](#) (vector1, vector2, theta_degrees)
Compute the angle in degrees between two vectors projected to 2D.
- subroutine [maths::rmsd](#) (rmsd_value, nb_atoms, coords1, coords2)
Compute the root-mean-square deviation (RMSD) between two sets of coordinates.
- subroutine [maths::calculate_cog](#) (cog, coords, nb_atoms)
Compute the center of geometry (centroid) of a set of points.
- subroutine [maths::calculate_distance](#) (p1, p2, dist)
Compute Euclidean distance between two 3D points.

12.9.1 Detailed Description

Mathematical and geometric utilities for clustering workflows.

Mathematical utilities and coordinate transformation routines.

This module provides fundamental math operations including cross products, coordinate transformations, angle calculations, and rotation matrices.

Author

Abraham Muñoz-Chicharro

Version

1.0

Date

2026-04-05

12.10 mod_array.f90 File Reference

Build/read/write encounter-based arrays and per-encounter metrics.

Modules

- module [mod_array](#)

Functions/Subroutines

- subroutine [mod_array::array_z_coord](#) (complexes, array_sorted, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute_crds)
Compute the Z-coordinate (along transformed z-axis) for each encounter and sort.
- subroutine [mod_array::array_atoms_dist](#) (complexes, array_sorted, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute1_crds, solute2_crds)
Compute minimum inter-atomic distance between two solutes for each encounter and sort.
- subroutine [mod_array::array_angle](#) (complexes, array_sorted, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, point1a, point1b, point2a, point2b, dimensions)
Compute angle between two vectors (defined by atom groups) per encounter and sort.
- subroutine [mod_array::write_array](#) (array, filename)
Write a numeric array to a formatted text file.
- subroutine [mod_array::read_array](#) (array, n, filename)
Read a numeric array from a formatted text file.
- subroutine [mod_array::merge_sorted_segments](#) (arr, complexes, tmp_arr, tmp_comp, left, mid, right)
Merge two adjacent sorted segments of arrays in place.

12.10.1 Detailed Description

Build/read/write encounter-based arrays and per-encounter metrics.

Provides routines to compute z-coordinate, distance, and angle-based arrays from encounter transformations, as well as array I/O and sorting helpers.

Author

Abraham Muñoz-Chicharro

Version

1.0

Date

2026-04-05

12.11 mod_assoc.f90 File Reference

Association/complexes file data structures and parsing routines.

Data Types

- type [mod_assoc::type_assoc_file](#)
Data type for storing binned data (eg.

Modules

- module [mod_assoc](#)

Functions/Subroutines

- subroutine `mod_assoc::allocate_assoc_object` (this, nlines)
Allocate storage for an association/complexes object.
- integer function `mod_assoc::size_assoc` (assoc_file)
Count encounter records in an association/complexes file.
- subroutine `mod_assoc::fill_assoc_object` (this, assoc_file)
Read an association/complexes file and populate `type_assoc_file`.
- subroutine `mod_assoc::write_complexes` (complexes, nb_encounters, complexes_filename)

12.11.1 Detailed Description

Association/complexes file data structures and parsing routines.

Association/complexes file utilities.

This module defines the `type_assoc_file` derived type and routines to allocate and populate it from an association/complexes text file. The `type_assoc_file` stores the raw file lines, header lines, per-encounter translation vectors and rotation vectors, and the two reference centers `xc1` and `xc2` extracted from the file header.

NOTE: The routines in this file are adapted from the SDA7 project's `mod_pdb.f90` (original author Neil Bruce). This file contains code and ideas derived from that implementation. Appropriate attribution to the SDA7 sources is retained here. Consult the SDA7 project for original authorship and licensing details.

Author

Abraham Muñoz-Chicharro

Version

1.0

Date

2026-04-05

12.12 mod_clust_algorithm.f90 File Reference

Hierarchical clustering algorithms and output helpers.

Modules

- module `mod_clust_algorithm`

Functions/Subroutines

- subroutine [mod_clust_algorithm::linkage_clustering_from_matrix](#) (matrix, n, linkage_type, output_name, complexes)
Perform hierarchical clustering on a distance matrix.
- subroutine [mod_clust_algorithm::linkage_clustering_from_array](#) (array, n, linkage_type, output_name, complexes)
Perform hierarchical clustering on a distance array.
- subroutine [mod_clust_algorithm::sort_complexes](#) (tot_encounters, cluster_indexes, cluster_count, active_clusters, representative_indexes, cluster_average, cluster_sd, array)
Sort clusters and associated arrays by a given key array.
- subroutine [mod_clust_algorithm::write_cluster_elements](#) (tot_encounters, cluster_parent, cluster_count, active_clusters, output_name)
Write cluster membership lists to a text file.
- subroutine [mod_clust_algorithm::write_clust_info](#) (tot_encounters, representative_indexes, active_clusters, clust_count, cluster_average, cluster_sd, output_name)
Write summary information about clusters to a text file.
- subroutine [mod_clust_algorithm::write_cluster_complexes](#) (tot_encounters, cluster_parent, cluster_count, active_clusters, output_name, complexes)
Write complexes contained in each cluster to individual files.

12.12.1 Detailed Description

Hierarchical clustering algorithms and output helpers.

12.13 mod_matrix.f90 File Reference

Build/read/write encounter-based matrices and arrays.

Modules

- module [mod_matrix](#)

Functions/Subroutines

- subroutine [mod_matrix::matrix_rmsd](#) (matrix, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, coord)
Build RMSD-based pairwise matrix between transformed coordinates.
- subroutine [mod_matrix::write_matrix](#) (matrix, filename)
Write a numeric matrix to a formatted text file.
- subroutine [mod_matrix::read_matrix](#) (matrix, n, filename)
Read a square numeric matrix from a formatted text file.

12.13.1 Detailed Description

Build/read/write encounter-based matrices and arrays.

Author

Abraham Muñoz-Chicharro

Version

1.0

Date

2026-04-05

12.14 mod_pdb.f90 File Reference

PDB data structures and parsing/allocation routines.

Data Types

- type [mod_pdb::type_pdb_file](#)
Data type for storing binned data (eg.
- type [mod_pdb::type_pdb_chain](#)
- type [mod_pdb::type_pdb_residue](#)
- type [mod_pdb::type_pdb_atom](#)

Modules

- module [mod_pdb](#)

Functions/Subroutines

- subroutine [mod_pdb::allocate_pdb_object](#) (this, pdb_file, natoms, nresidues, nchains)
Allocate and initialize a [type_pdb_file](#) structure from a PDB file.
- integer function [mod_pdb::count_atoms](#) (pdb_file)
Count ATOM/HETATM records in a PDB file.
- integer function [mod_pdb::count_residues](#) (pdb_file)
Count residues in a PDB file.
- integer function [mod_pdb::count_chains](#) (pdb_file)
Count chain separators (TER records) in a PDB file.
- subroutine [mod_pdb::fill_pdb_object](#) (this, pdb_file)
Parse PDB records and populate the [type_pdb_file](#) structure.

12.14.1 Detailed Description

PDB data structures and parsing/allocation routines.

Module to define a pdb type and pdb reading routine for tools.

NOTE: This implementation is based on the SDA7 project's [mod_pdb.f90](#) (original author Neil Bruce). The present file has been modified and expanded for the clustering tools in this workspace. Notable enhancements include the addition of `count_residues` and `count_chains` functions and changes that allow creation of richer in-memory objects where residues contain their atoms and chains contain residues and atoms. These changes make it possible to build `type_pdb_file` instances with proper `residues` and `chains` substructures (each storing their atoms) rather than storing atoms in a single flat list only.

The original SDA7 code and author attribution are retained; consult the SDA7 sources for original licensing and authorship details.

Author

Neil Bruce

Abraham Muñiz-Chicharro (modifications and extensions)

Version

1.0

Date

2026-04-05

12.15 mod_threshold.f90 File Reference

Utilities to compute threshold arrays from encounter complexes.

Modules

- module [mod_threshold](#)

Functions/Subroutines

- subroutine [mod_threshold::array_z_coord](#) (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute_crds)
Compute the Z-coordinate (along transformed z-axis) for each encounter.
- subroutine [mod_threshold::array_atoms_dist](#) (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, solute1_crds, solute2_crds)
Compute minimum inter-atomic distance between two solutes for each encounter.
- subroutine [mod_threshold::array_angle](#) (array, n, nb_atoms, xc1, xc2, trans_vector, rot1, rot2, point1a, point1b, point2a, point2b, dimensions)
Compute angle between two vectors (defined by atom groups) per encounter.
- subroutine [mod_threshold::sort_array](#) (arr, encounter_indexes)
Sort an array in ascending order and reorder encounter indexes accordingly.

12.15.1 Detailed Description

Utilities to compute threshold arrays from encounter complexes.

Author

Abraham Muñoz-Chicharro

Version

1.0

Date

2026-04-05

12.16 read_input.f90 File Reference

Input readers for PDB and association/complexes files.

Modules

- module [read_input](#)

Functions/Subroutines

- subroutine [read_input::read_pdb](#) (pdb, filename, tot_atoms, tot_residues, tot_chains)
Read a PDB file into a `type_pdb_file` object.
- subroutine [read_input::read_assoc](#) (assoc, filename, tot_encounters)
Read an association/complexes file into a `type_assoc_file` object.

12.16.1 Detailed Description

Input readers for PDB and association/complexes files.

Author

Abraham Muñoz-Chicharro

Version

1.0

Date

2026-04-05

12.17 threshold.f90 File Reference

Threshold-based encounter selection utility.

Functions/Subroutines

- program [main](#)
- subroutine [parse_arguments](#) (p_pdb1_filename, p_pdb1_bool, p_pdb2_filename, p_pdb2_bool, p_complexes_filename, p_complexes_bool, p_datadist_filename, p_array_bool, p_output_complexes_filename, p_output_complexes_bool, p_nb_encounters, p_nb_encounters_bool, p_cutoff, p_cutoff_bool, p_threshold_type, p_thresholdtype_bool, p_arr_atoms1a, p_arr_atoms1a_bool, p_arr_atoms1b, p_arr_atoms1b_bool, p_arr_atoms2a, p_arr_atoms2a_bool, p_arr_atoms2b, p_arr_atoms2b_bool, p_help_bool)
Parse command-line arguments for threshold.
- subroutine [print_help](#) (help_option)
Print usage/help text for threshold.
- subroutine [read_atoms_coord](#) (atoms_array, atoms_coord, tot_atoms, pdb, pdb_filename)
Read coordinates for a specified list of atoms (names or indices).
- subroutine [write_cutoff_complexes](#) (complexes_arg, distarray_arg, encounter_indexes_arg, nb_encounters_arg, cutoff_arg, output_name_arg)
Write selected encounter complexes to an output file.
- subroutine [write_array](#) (array_arg, filename_arg, cutoff_arg)
Write array values up to a cutoff to a formatted file.

12.17.1 Detailed Description

Threshold-based encounter selection utility.

[threshold.f90](#) - Select encounters based on threshold criteria.

Usage: ./threshold -help

Computes thresholds (z_coord, atoms_dist, angles) and writes selected encounters to output. See print_help for usage.

Author

Abraham Muñoz-Chicharro

Version

1.0

Date

2026-04-05

12.17.2 Function/Subroutine Documentation

12.17.2.1 main()

program main

12.17.2.2 parse_arguments()

```
subroutine main::parse_arguments (
    character*128, intent(out) p_pdb1_filename,
    logical, intent(out) p_pdb1_bool,
    character*128, intent(out) p_pdb2_filename,
    logical, intent(out) p_pdb2_bool,
    character*128, intent(out) p_complexes_filename,
    logical, intent(out) p_complexes_bool,
    character*128, intent(out) p_datadist_filename,
    logical, intent(out) p_array_bool,
    character*128, intent(out) p_output_complexes_filename,
    logical, intent(out) p_output_complexes_bool,
    integer, intent(out) p_nb_encounters,
    logical, intent(out) p_nb_encounters_bool,
    real (kind=8), intent(out) p_cutoff,
    logical, intent(out) p_cutoff_bool,
    character*128, intent(out) p_threshold_type,
    logical, intent(out) p_thresholdtype_bool,
    character*128, dimension(:), intent(out), allocatable p_arr_atoms1a,
    logical, intent(out) p_arr_atoms1a_bool,
    character*128, dimension(:), intent(out), allocatable p_arr_atoms1b,
    logical, intent(out) p_arr_atoms1b_bool,
    character*128, dimension(:), intent(out), allocatable p_arr_atoms2a,
    logical, intent(out) p_arr_atoms2a_bool,
    character*128, dimension(:), intent(out), allocatable p_arr_atoms2b,
    logical, intent(out) p_arr_atoms2b_bool,
    logical, intent(out) p_help_bool )
```

Parse command-line arguments for threshold.

Parses all CLI flags and fills program configuration values. Atom-group flags consume multiple tokens until the next flag.

Parameters

out	<i>p_pdb1_filename</i>	Value parsed from -pdb1
out	<i>p_pdb1_bool</i>	Whether -pdb1 was provided
out	<i>p_pdb2_filename</i>	Value parsed from -pdb2
out	<i>p_pdb2_bool</i>	Whether -pdb2 was provided
out	<i>p_complexes_filename</i>	Value parsed from -complexes
out	<i>p_complexes_bool</i>	Whether -complexes was provided
out	<i>p_datadist_filename</i>	Value parsed from -array
out	<i>p_array_bool</i>	Whether -array was provided
out	<i>p_output_complexes_filename</i>	Value parsed from -complexes_output
out	<i>p_output_complexes_bool</i>	Whether -complexes_output was provided
out	<i>p_nb_encounters</i>	Value parsed from -nb_encounters
out	<i>p_nb_encounters_bool</i>	Whether -nb_encounters was provided

Parameters

out	<i>p_cutoff</i>	Value parsed from <code>-cutoff</code>
out	<i>p_cutoff_bool</i>	Whether <code>-cutoff</code> was provided
out	<i>p_threshold_type</i>	Value parsed from <code>-threshold_type</code>
out	<i>p_thresholdtype_bool</i>	Whether <code>-threshold_type</code> was provided
out	<i>p_arr_atoms1a</i>	Atom list parsed from <code>-atoms1</code> or <code>-atoms1a</code>
out	<i>p_arr_atoms1a_bool</i>	Whether <code>-atoms1</code> or <code>-atoms1a</code> was provided
out	<i>p_arr_atoms1b</i>	Atom list parsed from <code>-atoms1b</code>
out	<i>p_arr_atoms1b_bool</i>	Whether <code>-atoms1b</code> was provided
out	<i>p_arr_atoms2a</i>	Atom list parsed from <code>-atoms2</code> or <code>-atoms2a</code>
out	<i>p_arr_atoms2a_bool</i>	Whether <code>-atoms2</code> or <code>-atoms2a</code> was provided
out	<i>p_arr_atoms2b</i>	Atom list parsed from <code>-atoms2b</code>
out	<i>p_arr_atoms2b_bool</i>	Whether <code>-atoms2b</code> was provided
out	<i>p_help_bool</i>	Whether <code>-help</code> was provided

12.17.2.3 `print_help()`

```
subroutine main::print_help (
    character(len=*), intent(in) help_option )
```

Print usage/help text for `threshold`.

Explains command-line options and provides examples per threshold type.

12.17.2.4 `read_atoms_coord()`

```
subroutine main::read_atoms_coord (
    character(len=*), dimension(:), intent(in) atoms_array,
    real(kind=8), dimension(:, :), intent(out), allocatable atoms_coord,
    integer, intent(in) tot_atoms,
    type ( type_pdb_file ), intent(in) pdb,
    character(len=*), intent(in) pdb_filename )
```

Read coordinates for a specified list of atoms (names or indices).

Accepts either atom indices or atom names. If atom names are provided, all matching atoms in the PDB are returned (in order).

Parameters

in	<i>atoms_array</i>	Array of atom name strings or index strings
out	<i>atoms_coord</i>	Allocated array with atom coordinates (n,3)
in	<i>tot_atoms</i>	Total number of atoms in <code>pdb</code>
in	<i>pdb</i>	PDB structure holding atom coordinates
in	<i>pdb_filename</i>	Reference filename for warnings

12.17.2.5 write_array()

```
subroutine main::write_array (
    real(kind=8), dimension(:), intent(in) array_arg,
    character*128, intent(in) filename_arg,
    real(kind=8), intent(in) cutoff_arg )
```

Write array values up to a cutoff to a formatted file.

Writes values from `array` (in ascending order) until `cutoff` is reached and stores them in `filename` using F10.4 formatting.

12.17.2.6 write_cutoff_complexes()

```
subroutine main::write_cutoff_complexes (
    type(type_assoc_file), intent(in) complexes_arg,
    real (kind=8), dimension(:), intent(in) distarray_arg,
    integer, dimension(:), intent(in) encounter_indexes_arg,
    integer, intent(in) nb_encounters_arg,
    real(kind=8), intent(in) cutoff_arg,
    character*128, intent(in) output_name_arg )
```

Write selected encounter complexes to an output file.

Writes the header lines from `complexeshead` and then appends encounter lines (from `complexeslines`) in the order provided by `encounter_indexes` until a value exceeding `cutoff` is encountered.

12.18 threshold.md File Reference

12.19 unit_test/README.md File Reference

12.20 unit_test/run_all_tests.sh File Reference

12.21 unit_test/test_array.f90 File Reference

Functions/Subroutines

- program [test_array](#)
Unit tests for array operations in the [mod_array](#) module.
- subroutine [test_write_read_array](#) (num_tests, num_passed, num_failed)
Unit test for [write_array](#) and [read_array](#) routines.
- subroutine [test_array_z_coord_sorting](#) (num_tests, num_passed, num_failed)
Unit test for [array_z_coord](#) sorting and complex reordering.
- subroutine [test_array_atoms_dist_sorting](#) (num_tests, num_passed, num_failed)
Unit test for [array_atoms_dist](#) sorting and complex reordering.
- subroutine [test_array_angle_sorting](#) (num_tests, num_passed, num_failed)
Unit test for [array_angle](#) sorting and complex reordering.
- subroutine [test_merge_sorted_segments](#) (num_tests, num_passed, num_failed)
Unit test for [merge_sorted_segments](#) routine.

12.21.1 Function/Subroutine Documentation

12.21.1.1 test_array()

```
program test_array
```

Unit tests for array operations in the [mod_array](#) module.

Tests include:

- Write and read array operations
- Z-coordinate array sorting with encounter metadata reordering
- Minimum-distance array sorting with encounter metadata reordering
- Angle array sorting with encounter metadata reordering
- Low-level merge of sorted segments with metadata reordering

Author

Abraham Muñoz-Chicharro

Version

1.0

12.21.1.2 test_array_angle_sorting()

```
subroutine test_array::test_array_angle_sorting (  
    integer, intent(inout) num_tests,  
    integer, intent(inout) num_passed,  
    integer, intent(inout) num_failed )
```

Unit test for array_angle sorting and complex reordering.

Author

Abraham

Version

1.0

Date

2026-04-06

Parameters

<i>in, out</i>	<i>num_tests</i>	Number of tests executed (incremented)
<i>in, out</i>	<i>num_passed</i>	Number of tests passed (incremented)
<i>in, out</i>	<i>num_failed</i>	Number of tests failed (incremented)

12.21.1.3 test_array_atoms_dist_sorting()

```
subroutine test_array::test_array_atoms_dist_sorting (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Unit test for array_atoms_dist sorting and complex reordering.

Author

Abraham

Version

1.0

Date

2026-04-06

Parameters

<i>in, out</i>	<i>num_tests</i>	Number of tests executed (incremented)
<i>in, out</i>	<i>num_passed</i>	Number of tests passed (incremented)
<i>in, out</i>	<i>num_failed</i>	Number of tests failed (incremented)

12.21.1.4 test_array_z_coord_sorting()

```
subroutine test_array::test_array_z_coord_sorting (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Unit test for array_z_coord sorting and complex reordering.

Author

Abraham

Version

1.0

Date

2026-04-06

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.21.1.5 test_merge_sorted_segments()

```
subroutine test_array::test_merge_sorted_segments (  
    integer, intent(inout) num_tests,  
    integer, intent(inout) num_passed,  
    integer, intent(inout) num_failed )
```

Unit test for merge_sorted_segments routine.

Author

Abraham

Version

1.0

Date

2026-04-06

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.21.1.6 test_write_read_array()

```
subroutine test_array::test_write_read_array (  
    integer, intent(inout) num_tests,
```

```
integer, intent(inout) num_passed,
integer, intent(inout) num_failed )
```

Unit test for write_array and read_array routines.

Author

Abraham

Version

1.0

Date

2026-04-06

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.22 unit_test/test_assoc.f90 File Reference

Functions/Subroutines

- program [test_assoc](#)
Unit tests for reading association/complexes files.
- subroutine [test_read_assoc_file](#) (num_tests, num_passed, num_failed)
Unit test for read_assoc file operations.
- subroutine [test_allocate_assoc_object](#) (num_tests, num_passed, num_failed)
Unit test for allocate_assoc_object routine.
- subroutine [test_size_assoc](#) (num_tests, num_passed, num_failed)
Unit test for size_assoc routine (line counting in assoc files).

12.22.1 Function/Subroutine Documentation

12.22.1.1 test_allocate_assoc_object()

```
subroutine test_assoc::test_allocate_assoc_object (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Unit test for allocate_assoc_object routine.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.22.1.2 test_assoc()

```
program test_assoc
```

Unit tests for reading association/complexes files.

Comprehensive test suite for reading and parsing association/complexes files in the [read_input](#) and [mod_assoc](#) modules including:

- **File reading:**
 - Valid assoc file reading
 - File existence checking
 - Error handling for missing files
 - Correct number of encounters parsed
- **Data structure allocation:**
 - Allocate assoc objects with correct sizes
 - Initialize translation vectors to zero
 - Initialize rotation vectors (rot1, rot2) to zero
 - Allocate lines array
- **File parsing:**
 - Skip comment lines (starting with #)
 - Count data lines correctly

Author

Abraham Muñoz-Chicharro

Version

1.0

- Extract center coordinates (xc1, xc2)
- Parse translation vectors
- Parse rotation vectors
- **Data integrity:**
 - Verify allocations match encounter count
 - Check header information is stored
 - Verify vector values are reasonable
 - Ensure no data corruption during read

Compile with: gfortran -o test_assoc [test_assoc.f90](#) ../src/read_input.f90 ../src/mod_assoc.f90

Author

Abraham Muñoz-Chicharro

Version

1.0

12.22.1.3 test_read_assoc_file()

```
subroutine test_assoc::test_read_assoc_file (  
    integer, intent(inout) num_tests,  
    integer, intent(inout) num_passed,  
    integer, intent(inout) num_failed )
```

Unit test for read_assoc file operations.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.22.1.4 test_size_assoc()

```

subroutine test_assoc::test_size_assoc (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )

```

Unit test for size_assoc routine (line counting in assoc files).

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.23 unit_test/test_clustering.f90 File Reference

Unit tests for clustering algorithms in [mod_clust_algorithm](#).

Functions/Subroutines

- program [test_clustering](#)
- subroutine [test_clustering_statistics](#) (num_tests, num_passed, num_failed)
Test clustering statistics calculations (mean, stddev)
- subroutine [test_simple_clustering](#) (num_tests, num_passed, num_failed)

- Test simple clustering scenarios (tight clusters, identical points)*
- subroutine [test_minimum_linkage](#) (num_tests, num_passed, num_failed)
Test minimum linkage clustering on 1D array.
- subroutine [test_maximum_linkage](#) (num_tests, num_passed, num_failed)
Test maximum linkage clustering on 1D array.
- subroutine [test_mean_linkage](#) (num_tests, num_passed, num_failed)
Test mean linkage clustering on 1D array.
- subroutine [test_cluster_writing](#) (num_tests, num_passed, num_failed)
Test writing cluster elements to file.

12.23.1 Detailed Description

Unit tests for clustering algorithms in [mod_clust_algorithm](#).

Tests include:

- Sort complexes functionality
- Write cluster elements
- Basic clustering threshold calculation
- Minimum linkage clustering
- Maximum linkage clustering
- Mean linkage clustering

Note: Array parameter is mandatory for all clustering methods EXCEPT RMSD. For RMSD-based clustering, array is optional. When provided, array is used to compute cluster averages and standard deviations.

Compile with: gfortran -fopenmp -o test_clustering [test_clustering.f90](#) ../src/mod_clust_algorithm.f90 ../src/read_↵ input.f90

Author

Abraham Muniz-Chicharro

Version

1.0

Date

2026-04-05

12.23.2 Function/Subroutine Documentation

12.23.2.1 test_cluster_writing()

```
subroutine test_clustering::test_cluster_writing (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Test writing cluster elements to file.

Parameters

in, out	<i>num_tests</i>	Number of tests run
in, out	<i>num_passed</i>	Number of tests passed
in, out	<i>num_failed</i>	Number of tests failed

12.23.2.2 test_clustering()

```
program test_clustering
```

12.23.2.3 test_clustering_statistics()

```
subroutine test_clustering::test_clustering_statistics (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Test clustering statistics calculations (mean, stddev)

Parameters

in, out	<i>num_tests</i>	Number of tests run
in, out	<i>num_passed</i>	Number of tests passed
in, out	<i>num_failed</i>	Number of tests failed

12.23.2.4 test_maximum_linkage()

```
subroutine test_clustering::test_maximum_linkage (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Test maximum linkage clustering on 1D array.

Parameters

in, out	<i>num_tests</i>	Number of tests run
in, out	<i>num_passed</i>	Number of tests passed
in, out	<i>num_failed</i>	Number of tests failed

12.23.2.5 test_mean_linkage()

```
subroutine test_clustering::test_mean_linkage (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Test mean linkage clustering on 1D array.

Parameters

in, out	<i>num_tests</i>	Number of tests run
in, out	<i>num_passed</i>	Number of tests passed
in, out	<i>num_failed</i>	Number of tests failed

12.23.2.6 test_minimum_linkage()

```
subroutine test_clustering::test_minimum_linkage (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Test minimum linkage clustering on 1D array.

Parameters

in, out	<i>num_tests</i>	Number of tests run
in, out	<i>num_passed</i>	Number of tests passed
in, out	<i>num_failed</i>	Number of tests failed

12.23.2.7 test_simple_clustering()

```
subroutine test_clustering::test_simple_clustering (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Test simple clustering scenarios (tight clusters, identical points)

Parameters

in, out	<i>num_tests</i>	Number of tests run
in, out	<i>num_passed</i>	Number of tests passed
in, out	<i>num_failed</i>	Number of tests failed

12.24 unit_test/test_maths.f90 File Reference

Functions/Subroutines

- program [test_maths](#)
Unit tests for mathematical operations in the maths module.
- subroutine [test_cross_product](#) (num_tests, num_passed, num_failed)
Unit test for cross product routine.
- subroutine [test_update_complex](#) (num_tests, num_passed, num_failed)
Unit test for update_complex routine.
- subroutine [test_vectors_angle_3d](#) (num_tests, num_passed, num_failed)
Unit test for vectors_angle_3D routine.
- subroutine [test_vectors_angle_2d](#) (num_tests, num_passed, num_failed)
Unit test for vectors_angle_2D routine.
- subroutine [test_rmsd](#) (num_tests, num_passed, num_failed)
Unit test for RMSD calculation routine.
- subroutine [test_calculate_cog](#) (num_tests, num_passed, num_failed)
Unit test for calculate_cog routine.
- subroutine [test_calculate_distance](#) (num_tests, num_passed, num_failed)
Unit test for calculate_distance routine.
- logical function [assert_vector_equal](#) (v1, v2, tolerance)
Helper function: Check if two vectors are approximately equal.

12.24.1 Function/Subroutine Documentation

12.24.1.1 assert_vector_equal()

```
logical function test_maths::assert_vector_equal (
    real(kind=8), dimension(3), intent(in) v1,
    real(kind=8), dimension(3), intent(in) v2,
    real(kind=8), intent(in) tolerance )
```

Helper function: Check if two vectors are approximately equal.

12.24.1.2 test_calculate_cog()

```
subroutine test_maths::test_calculate_cog (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Unit test for calculate_cog routine.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.24.1.3 test_calculate_distance()

```
subroutine test_maths::test_calculate_distance (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Unit test for calculate_distance routine.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.24.1.4 test_cross_product()

```
subroutine test_maths::test_cross_product (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Unit test for cross product routine.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.24.1.5 test_maths()

```
program test_maths
```

Unit tests for mathematical operations in the maths module.

Comprehensive test suite for all subroutines in the maths module including:

- **Cross product:**
 - Basis vectors ($i \times j = k$)
 - Reverse order (anticommutative property)
 - Self cross product ($v \times v = 0$)
 - Arbitrary vectors
- **Coordinate transformations (update_complex):**
 - Identity transformations
 - Pure translation
 - Only xc1 translation
 - Only xc2 translation
 - Only rotation (90 degrees)
 - Combined transformations
- **3D angle calculations (vectors_angle_3D):**
 - Parallel vectors (0 degrees)

Author

Abraham Muñiz-Chicharro

Version

1.0

- Perpendicular vectors (90 degrees)
- Opposite vectors (180 degrees)
- 45 degree angles

- **2D angle calculations (vectors_angle_2D):**
 - Parallel vectors in XY plane (0 degrees)
 - Perpendicular vectors in XY plane (90 degrees)
 - Opposite vectors in XY plane (180 degrees)
 - Zero magnitude vectors handling

- **RMSD calculation:**
 - Identical coordinates (RMSD = 0)
 - Simple translations
 - Known RMSD values

- **Center of geometry (calculate_cog):**
 - Single point
 - Points at origin
 - Symmetric point distributions
 - General cases

- **Distance calculation (calculate_distance):**
 - Distance to self (0)
 - Unit distances
 - 3D diagonal distances
 - Arbitrary point pairs

Compile with: gfortran -o test_maths [test_maths.f90](#) ../src/maths.f90 ../src/mod_pdb.f90

Author

Abraham Muñiz-Chicharro

Version

1.0

12.24.1.6 test_rmsd()

```
subroutine test_maths::test_rmsd (  
    integer, intent(inout) num_tests,  
    integer, intent(inout) num_passed,  
    integer, intent(inout) num_failed )
```

Unit test for RMSD calculation routine.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.24.1.7 test_update_complex()

```
subroutine test_maths::test_update_complex (  
    integer, intent(inout) num_tests,  
    integer, intent(inout) num_passed,  
    integer, intent(inout) num_failed )
```

Unit test for update_complex routine.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.24.1.8 test_vectors_angle_2d()

```
subroutine test_maths::test_vectors_angle_2d (  
    integer, intent(inout) num_tests,  
    integer, intent(inout) num_passed,  
    integer, intent(inout) num_failed )
```

Unit test for vectors_angle_2D routine.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.24.1.9 test_vectors_angle_3d()

```
subroutine test_maths::test_vectors_angle_3d (  
    integer, intent(inout) num_tests,  
    integer, intent(inout) num_passed,  
    integer, intent(inout) num_failed )
```

Unit test for vectors_angle_3D routine.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.25 unit_test/test_matrix.f90 File Reference

Functions/Subroutines

- program [test_matrix](#)
Unit tests for matrix operations in the [mod_matrix](#) module.
- subroutine [test_write_read_matrix](#) (num_tests, num_passed, num_failed)
Unit test for write_matrix and read_matrix routines.
- subroutine [test_matrix_rmsd_calc](#) (num_tests, num_passed, num_failed)
Unit test for matrix_rmsd routine.

12.25.1 Function/Subroutine Documentation

12.25.1.1 test_matrix()

```
program test_matrix
```

Unit tests for matrix operations in the [mod_matrix](#) module.

Tests include:

- Write and read matrix operations
- RMSD matrix calculation

Compile with: gfortran -fopenmp -o test_matrix [test_matrix.f90](#) ../src/mod_matrix.f90 ../src/mathsf90 ../src/mod_↵
pdb.f90

Author

Abraham Muñoz-Chicharro

Version

1.0

12.25.1.2 test_matrix_rmsd_calc()

```
subroutine test_matrix::test_matrix_rmsd_calc (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Unit test for matrix_rmsd routine.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.25.1.3 test_write_read_matrix()

```
subroutine test_matrix::test_write_read_matrix (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Unit test for write_matrix and read_matrix routines.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.26 unit_test/test_pdb.f90 File Reference

Functions/Subroutines

- program [test_pdb](#)
Unit tests for PDB operations in the [mod_pdb](#) module.
- subroutine [test_pdb_allocation](#) (num_tests, num_passed, num_failed)
Unit test for PDB allocation and initialization routines.
- subroutine [test_atom_properties](#) (num_tests, num_passed, num_failed)
Unit test for atom property assignment in PDB structure.
- subroutine [test_residue_management](#) (num_tests, num_passed, num_failed)
Unit test for residue management in PDB structure.
- subroutine [test_chain_management](#) (num_tests, num_passed, num_failed)
Unit test for chain management in PDB structure.
- subroutine [test_mod_pdb_routines](#) (num_tests, num_passed, num_failed)
Unit test for [mod_pdb](#) routines (count/allocate/fill).

12.26.1 Function/Subroutine Documentation

12.26.1.1 test_atom_properties()

```
subroutine test_pdb::test_atom_properties (  
    integer, intent(inout) num_tests,  
    integer, intent(inout) num_passed,  
    integer, intent(inout) num_failed )
```

Unit test for atom property assignment in PDB structure.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.26.1.2 test_chain_management()

```
subroutine test_pdb::test_chain_management (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Unit test for chain management in PDB structure.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.26.1.3 test_mod_pdb_routines()

```
subroutine test_pdb::test_mod_pdb_routines (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Unit test for [mod_pdb](#) routines (count/allocate/fill).

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.26.1.4 test_pdb()

```
program test_pdb
```

Unit tests for PDB operations in the [mod_pdb](#) module.

Tests include:

- PDB type allocation and initialization
- Atom and residue management
- Residue structure handling
- Reading PDB files and filling the PDB data structure

Compile with: gfortran -o test_pdb [test_pdb.f90](#) [mod_pdb.f90](#)

Author

Abraham Muñiz-Chicharro

Version

1.0

12.26.1.5 test_pdb_allocation()

```
subroutine test_pdb::test_pdb_allocation (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Unit test for PDB allocation and initialization routines.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.26.1.6 test_residue_management()

```
subroutine test_pdb::test_residue_management (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Unit test for residue management in PDB structure.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.27 unit_test/test_read_input.f90 File Reference

Functions/Subroutines

- program [test_read_input](#)
Unit tests for the [read_input](#) module.
- subroutine [test_read_input_assoc](#) (num_tests, num_passed, num_failed)
Unit test for reading association files via [read_input](#) module.
- subroutine [test_error_handling](#) (num_tests, num_passed, num_failed)
Unit test for error handling in [read_input](#) module.

12.27.1 Function/Subroutine Documentation

12.27.1.1 test_error_handling()

```
subroutine test_read_input::test_error_handling (  
    integer, intent(inout) num_tests,  
    integer, intent(inout) num_passed,  
    integer, intent(inout) num_failed )
```

Unit test for error handling in [read_input](#) module.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.27.1.2 test_read_input()

program test_read_input

Unit tests for the [read_input](#) module.

Comprehensive test suite for reading PDB and association/complexes files using the [read_input](#) module wrapper functions including:

- **PDB file reading (read_pdb):**
 - Valid PDB file reading
 - File existence checking
 - Atom count verification
 - Residue count verification
 - Chain count verification
 - Proper structure allocation
 - Error handling for missing files
- **Association file reading (read_assoc):**
 - Valid assoc file reading
 - Encounter count verification
 - Structure allocation and initialization
 - Vector data integrity
 - Error handling for missing files

Author

Abraham Muñoz-Chicharro

Version

1.0

- **Integration tests:**
 - Sequential reading of multiple files
 - Proper cleanup between reads
 - Memory management

Compile with: gfortran -o test_read_input [test_read_input.f90](#) ../src/read_input.f90 ../src/mod_pdb.f90 ../src/mod_↵_assoc.f90

Author

Abraham Muñoz-Chicharro

Version

1.0

12.27.1.3 test_read_input_assoc()

```
subroutine test_read_input::test_read_input_assoc (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Unit test for reading association files via [read_input](#) module.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.28 unit_test/test_threshold.f90 File Reference

Functions/Subroutines

- program [test_threshold](#)
Unit tests for threshold operations in the [mod_threshold](#) module.
- subroutine [test_array_z_coord](#) (num_tests, num_passed, num_failed)
Unit test for array_z_coord routine.
- subroutine [test_array_atoms_dist](#) (num_test, num_passed, num_failed)
Unit test for array_atoms_dist routine.
- subroutine [test_array_angle](#) (num_tests, num_passed, num_failed)
Unit test for array_angle routine.
- subroutine [test_sort_array](#) (num_tests, num_passed, num_failed)
Unit test for sort_array routine.

12.28.1 Function/Subroutine Documentation

12.28.1.1 test_array_angle()

```
subroutine test_threshold::test_array_angle (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Unit test for array_angle routine.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.28.1.2 test_array_atoms_dist()

```
subroutine test_threshold::test_array_atoms_dist (
    integer, intent(inout) num_tests,
    integer, intent(inout) num_passed,
    integer, intent(inout) num_failed )
```

Unit test for array_atoms_dist routine.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.28.1.3 test_array_z_coord()

```
subroutine test_threshold::test_array_z_coord (  
    integer, intent(inout) num_tests,  
    integer, intent(inout) num_passed,  
    integer, intent(inout) num_failed )
```

Unit test for array_z_coord routine.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

in, out	<i>num_tests</i>	Number of tests executed (incremented)
in, out	<i>num_passed</i>	Number of tests passed (incremented)
in, out	<i>num_failed</i>	Number of tests failed (incremented)

12.28.1.4 test_sort_array()

```
subroutine test_threshold::test_sort_array (  
    integer, intent(inout) num_tests,  
    integer, intent(inout) num_passed,  
    integer, intent(inout) num_failed )
```

Unit test for sort_array routine.

Author

Abraham

Version

1.0

Date

2024-06-09

Parameters

<i>in, out</i>	<i>num_tests</i>	Number of tests executed (incremented)
<i>in, out</i>	<i>num_passed</i>	Number of tests passed (incremented)
<i>in, out</i>	<i>num_failed</i>	Number of tests failed (incremented)

12.28.1.5 test_threshold()

```
program test_threshold
```

Unit tests for threshold operations in the [mod_threshold](#) module.

Tests include:

- Z-coordinate array calculation
- Minimum distance calculation between atoms
- Angle computation for different dimensions
- Array sorting with encounter index tracking

Compile with: `gfortran -fopenmp -o test_threshold test\_threshold.f90 ../src/mod_threshold.f90 ../src/mathsf90 ../src/mod_pdb.f90`

Author

Abraham Muñiz-Chicharro

Version

1.0

Index

- allocate_assoc_object
 - mod_assoc, 26
- allocate_pdb_object
 - mod_pdb, 33
- analyze_residues.f90, 49
 - main, 50
 - print_help, 50
- analyze_residues.md, 50
- array_angle
 - mod_array, 23
 - mod_threshold, 36
- array_atoms_dist
 - mod_array, 23
 - mod_threshold, 37
- array_z_coord
 - mod_array, 24
 - mod_threshold, 37
- assert_vector_equal
 - test_maths.f90, 79
- atoms
 - mod_pdb::type_pdb_file, 45
 - mod_pdb::type_pdb_residue, 46
- beta
 - mod_pdb::type_pdb_atom, 43
- calculate_cog
 - maths, 19
- calculate_distance
 - maths, 20
- chainid
 - mod_pdb::type_pdb_atom, 43
 - mod_pdb::type_pdb_residue, 47
- chains
 - mod_pdb::type_pdb_file, 45
- clust.f90, 50
 - main, 51
 - parse_arguments, 51
 - print_help, 51
- clust.md, 52
- clust_all
 - clust_all.f90, 52
- clust_all.f90, 52
 - clust_all, 52
 - parse_arguments, 53
 - print_help, 54
 - read_atoms_coord, 54
- clust_all.md, 55
- coord
 - mod_pdb::type_pdb_atom, 43
- count_atoms
 - mod_pdb, 34
- count_chains
 - mod_pdb, 34
- count_residues
 - mod_pdb, 35
- cross
 - maths, 20
- fill_assoc_object
 - mod_assoc, 27
- fill_pdb_object
 - mod_pdb, 35
- head
 - mod_assoc::type_assoc_file, 41
- id
 - mod_pdb::type_pdb_atom, 43
- lines
 - mod_assoc::type_assoc_file, 42
- linkage_clustering_from_array
 - mod_clust_algorithm, 28
- linkage_clustering_from_matrix
 - mod_clust_algorithm, 29
- main
 - analyze_residues.f90, 50
 - clust.f90, 51
 - make_data.f90, 55
 - threshold.f90, 65
- make_data.f90, 55
 - main, 55
 - parse_arguments, 56
 - print_help, 57
 - read_atoms_coord, 57
- make_data.md, 57
- maths, 19
 - calculate_cog, 19
 - calculate_distance, 20
 - cross, 20
 - rmsd, 20
 - update_complex, 21
 - vectors_angle_2d, 21
 - vectors_angle_3d, 22
- maths.f90, 57
- matrix_rmsd
 - mod_matrix, 32
- merge_sorted_segments
 - mod_array, 25

- mod_array, 22
 - array_angle, 23
 - array_atoms_dist, 23
 - array_z_coord, 24
 - merge_sorted_segments, 25
 - read_array, 25
 - write_array, 26
- mod_array.f90, 58
- mod_assoc, 26
 - allocate_assoc_object, 26
 - fill_assoc_object, 27
 - size_assoc, 27
 - write_complexes, 28
- mod_assoc.f90, 59
- mod_assoc::type_assoc_file, 41
 - head, 41
 - lines, 42
 - nlines, 42
 - rot1, 42
 - rot2, 42
 - trans_vector, 42
 - xc1, 42
 - xc2, 42
- mod_clust_algorithm, 28
 - linkage_clustering_from_array, 28
 - linkage_clustering_from_matrix, 29
 - sort_complexes, 29
 - write_clust_info, 30
 - write_cluster_complexes, 30
 - write_cluster_elements, 31
- mod_clust_algorithm.f90, 60
- mod_matrix, 31
 - matrix_rmsd, 32
 - read_matrix, 32
 - write_matrix, 33
- mod_matrix.f90, 61
- mod_pdb, 33
 - allocate_pdb_object, 33
 - count_atoms, 34
 - count_chains, 34
 - count_residues, 35
 - fill_pdb_object, 35
- mod_pdb.f90, 62
- mod_pdb::type_pdb_atom, 43
 - beta, 43
 - chainid, 43
 - coord, 43
 - id, 43
 - name, 44
 - occ, 44
 - record, 44
 - resid, 44
 - resname, 44
- mod_pdb::type_pdb_chain, 44
- mod_pdb::type_pdb_file, 45
 - atoms, 45
 - chains, 45
 - natoms, 45
 - nchains, 45
 - nresidues, 46
 - residues, 46
- mod_pdb::type_pdb_residue, 46
 - atoms, 46
 - chainid, 47
 - natoms, 47
 - resid, 47
 - resname, 47
- mod_threshold, 36
 - array_angle, 36
 - array_atoms_dist, 37
 - array_z_coord, 37
 - sort_array, 38
- mod_threshold.f90, 63
- name
 - mod_pdb::type_pdb_atom, 44
- natoms
 - mod_pdb::type_pdb_file, 45
 - mod_pdb::type_pdb_residue, 47
- nchains
 - mod_pdb::type_pdb_file, 45
- nlines
 - mod_assoc::type_assoc_file, 42
- nresidues
 - mod_pdb::type_pdb_file, 46
- occ
 - mod_pdb::type_pdb_atom, 44
- parse_arguments
 - clust.f90, 51
 - clust_all.f90, 53
 - make_data.f90, 56
 - threshold.f90, 66
- print_help
 - analyze_residues.f90, 50
 - clust.f90, 51
 - clust_all.f90, 54
 - make_data.f90, 57
 - threshold.f90, 67
- read_array
 - mod_array, 25
- read_assoc
 - read_input, 38
- read_atoms_coord
 - clust_all.f90, 54
 - make_data.f90, 57
 - threshold.f90, 67
- read_input, 38
 - read_assoc, 38
 - read_pdb, 39
- read_input.f90, 64
- read_matrix
 - mod_matrix, 32
- read_pdb
 - read_input, 39

- record
 - mod_pdb::type_pdb_atom, [44](#)
- resid
 - mod_pdb::type_pdb_atom, [44](#)
 - mod_pdb::type_pdb_residue, [47](#)
- residues
 - mod_pdb::type_pdb_file, [46](#)
- resname
 - mod_pdb::type_pdb_atom, [44](#)
 - mod_pdb::type_pdb_residue, [47](#)
- rmsd
 - maths, [20](#)
- rot1
 - mod_assoc::type_assoc_file, [42](#)
- rot2
 - mod_assoc::type_assoc_file, [42](#)
- size_assoc
 - mod_assoc, [27](#)
- sort_array
 - mod_threshold, [38](#)
- sort_complexes
 - mod_clust_algorithm, [29](#)
- test_allocate_assoc_object
 - test_assoc.f90, [72](#)
- test_array
 - test_array.f90, [69](#)
- test_array.f90
 - test_array, [69](#)
 - test_array_angle_sorting, [69](#)
 - test_array_atoms_dist_sorting, [70](#)
 - test_array_z_coord_sorting, [70](#)
 - test_merge_sorted_segments, [71](#)
 - test_write_read_array, [71](#)
- test_array_angle
 - test_threshold.f90, [93](#)
- test_array_angle_sorting
 - test_array.f90, [69](#)
- test_array_atoms_dist
 - test_threshold.f90, [94](#)
- test_array_atoms_dist_sorting
 - test_array.f90, [70](#)
- test_array_z_coord
 - test_threshold.f90, [95](#)
- test_array_z_coord_sorting
 - test_array.f90, [70](#)
- test_assoc
 - test_assoc.f90, [73](#)
- test_assoc.f90
 - test_allocate_assoc_object, [72](#)
 - test_assoc, [73](#)
 - test_read_assoc_file, [74](#)
 - test_size_assoc, [75](#)
- test_atom_properties
 - test_pdb.f90, [87](#)
- test_calculate_cog
 - test_maths.f90, [79](#)
- test_calculate_distance
 - test_maths.f90, [80](#)
- test_chain_management
 - test_pdb.f90, [88](#)
- test_cluster_writing
 - test_clustering.f90, [76](#)
- test_clustering
 - test_clustering.f90, [77](#)
- test_clustering.f90
 - test_cluster_writing, [76](#)
 - test_clustering, [77](#)
 - test_clustering_statistics, [77](#)
 - test_maximum_linkage, [77](#)
 - test_mean_linkage, [77](#)
 - test_minimum_linkage, [78](#)
 - test_simple_clustering, [78](#)
- test_clustering_statistics
 - test_clustering.f90, [77](#)
- test_cross_product
 - test_maths.f90, [80](#)
- test_error_handling
 - test_read_input.f90, [91](#)
- test_maths
 - test_maths.f90, [81](#)
- test_maths.f90
 - assert_vector_equal, [79](#)
 - test_calculate_cog, [79](#)
 - test_calculate_distance, [80](#)
 - test_cross_product, [80](#)
 - test_maths, [81](#)
 - test_rmsd, [82](#)
 - test_update_complex, [83](#)
 - test_vectors_angle_2d, [84](#)
 - test_vectors_angle_3d, [84](#)
- test_matrix
 - test_matrix.f90, [85](#)
- test_matrix.f90
 - test_matrix, [85](#)
 - test_matrix_rmsd_calc, [85](#)
 - test_write_read_matrix, [86](#)
- test_matrix_rmsd_calc
 - test_matrix.f90, [85](#)
- test_maximum_linkage
 - test_clustering.f90, [77](#)
- test_mean_linkage
 - test_clustering.f90, [77](#)
- test_merge_sorted_segments
 - test_array.f90, [71](#)
- test_minimum_linkage
 - test_clustering.f90, [78](#)
- test_mod_pdb_routines
 - test_pdb.f90, [88](#)
- test_pdb
 - test_pdb.f90, [89](#)
- test_pdb.f90
 - test_atom_properties, [87](#)
 - test_chain_management, [88](#)
 - test_mod_pdb_routines, [88](#)
 - test_pdb, [89](#)

- test_pdb_allocation, 89
 - test_residue_management, 90
- test_pdb_allocation
 - test_pdb.f90, 89
- test_read_assoc_file
 - test_assoc.f90, 74
- test_read_input
 - test_read_input.f90, 92
- test_read_input.f90
 - test_error_handling, 91
 - test_read_input, 92
 - test_read_input_assoc, 92
- test_read_input_assoc
 - test_read_input.f90, 92
- test_residue_management
 - test_pdb.f90, 90
- test_rmsd
 - test_maths.f90, 82
- test_simple_clustering
 - test_clustering.f90, 78
- test_size_assoc
 - test_assoc.f90, 75
- test_sort_array
 - test_threshold.f90, 95
- test_threshold
 - test_threshold.f90, 96
- test_threshold.f90
 - test_array_angle, 93
 - test_array_atoms_dist, 94
 - test_array_z_coord, 95
 - test_sort_array, 95
 - test_threshold, 96
- test_update_complex
 - test_maths.f90, 83
- test_vectors_angle_2d
 - test_maths.f90, 84
- test_vectors_angle_3d
 - test_maths.f90, 84
- test_write_read_array
 - test_array.f90, 71
- test_write_read_matrix
 - test_matrix.f90, 86
- threshold.f90, 65
 - main, 65
 - parse_arguments, 66
 - print_help, 67
 - read_atoms_coord, 67
 - write_array, 68
 - write_cutoff_complexes, 68
- threshold.md, 68
- trans_vector
 - mod_assoc::type_assoc_file, 42
- unit_test/README.md, 68
- unit_test/run_all_tests.sh, 68
- unit_test/test_array.f90, 68
- unit_test/test_assoc.f90, 72
- unit_test/test_clustering.f90, 75
- unit_test/test_maths.f90, 79
- unit_test/test_matrix.f90, 85
- unit_test/test_pdb.f90, 87
- unit_test/test_read_input.f90, 91
- unit_test/test_threshold.f90, 93
- update_complex
 - maths, 21
- vectors_angle_2d
 - maths, 21
- vectors_angle_3d
 - maths, 22
- write_array
 - mod_array, 26
 - threshold.f90, 68
- write_clust_info
 - mod_clust_algorithm, 30
- write_cluster_complexes
 - mod_clust_algorithm, 30
- write_cluster_elements
 - mod_clust_algorithm, 31
- write_complexes
 - mod_assoc, 28
- write_cutoff_complexes
 - threshold.f90, 68
- write_matrix
 - mod_matrix, 33
- xc1
 - mod_assoc::type_assoc_file, 42
- xc2
 - mod_assoc::type_assoc_file, 42